Software-as-a-Graph: Heterogeneous Graph Learning for Pre-Deployment Dependability Analysis of Complex Distributed Systems

Ibrahim Onuralp Yigit[a,*], Feza Buzluca[a]

[a]*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Modern asynchronous, event-driven architectures introduce a critical pre-deployment visibility barrier: bug-free services can still suffer catastrophic cascading outages from hidden single points of failure and mismatched middleware Quality-of-Service (QoS) contracts—the Architecture–Code Gap. To evaluate dependability before deployment without runtime telemetry, we present Software-as-a-Graph (SaG), a Static System Analysis framework transforming Architecture-as-Code manifests into typed multigraphs. SaG pairs a relation-specific Heterogeneous Graph Transformer with QoS edge encodings (HGT-QoS) for cascade blast-radius forecasting with an interpretable ISO/IEC 25010 attribution layer guiding repairs.

Evaluated across twelve complex synthetic architectures (2,461 components) and five open-source systems (351 components), relation typing yields a robust inductive bias under distribution shift ($\Delta\rho = +0.114$, $p = 0.0122$, winning 11 of 12 folds), with QoS encodings contributing $+0.054$ ($p = 0.0093$). Against training-free QoS-weighted centrality, learned ranking does not establish a statistically significant advantage ($+0.127$, $p = 0.077$). Zero-shot transfer achieves $\rho = 0.680$ overall, but narrows to $+0.160$ on active failure-propagating components, reflecting a domain boundary on synchronous microservice call trees. Regarding computational sustainability, neural inference requires only 56 ms (0.02% of pipeline time), eliminating energy-intensive staging cluster provisioning, while deterministic graph feature extraction dominates execution time. In complex distributed systems, deep heterogeneous learning is justified not for scalar ranking alone, but for its relational inductive bias, attention-driven channel diagnosis, and edge-level criticality prediction.

Keywords: Heterogeneous graph neural networks, Distributed systems dependability, Publish–subscribe architecture, Cascading failures, Static system analysis, Explainable AI

1. Introduction

1.1. Motivation

Modern large-scale distributed software systems increasingly rely on asynchronous, event-driven, and publish–subscribe (pub-sub) architectures to satisfy demanding scalability and throughput requirements. Across domains including autonomous driving (ROS 2 [1]), enterprise event streaming (Apache Kafka [2]), cyber-physical systems (DDS [3]), IoT fleets (MQTT [4]), cloud-native microservices [5, 6], and distributed AI/LLM serving clusters, pub-sub decouples communicating components in space, time, and synchronization [7]. Components interact indirectly through intermediate topics and brokers without maintaining static caller–callee references. Furthermore, modern middleware specifications allow engineers to configure deployment-time Quality-of-Service (QoS) policies—governing transport reliability, durability, priorities, and deadlines—to shape traffic behavior under peak load and network stress.

While architectural decoupling provides elastic scalability, it creates a critical **visibility barrier** for system performance, reliability, and computational sustainability:

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

- **Indirect Degradation Pathways:** Unlike synchronous architectures where interactions follow explicit call graphs, asynchronous event meshes decouple publishers and subscribers. Cascading outages, buffer saturation, head-of-line blocking, and backpressure propagate across multi-hop logical paths spanning brokers, shared topics, colocated hosts, and shared runtime libraries [8, 9].
- **Heterogeneous Degradation Mechanisms:** Disturbances propagate through distinct mechanisms: *sequential cascades* (e.g., slow subscribers inducing queue congestion and upstream publisher throttling [10]) or *simultaneous blast radii* (e.g., shared library crashes or host hardware failures instantaneously terminating all colocated services). Conventional call graphs fail to represent these layered dependencies.

Detecting these vulnerabilities is most effective and cost-efficient **prior to deployment**, during architecture design and continuous integration (CI/CD), adhering to foundational principles of dependable computing [11, 12]. However, during pre-deployment stages, **no runtime telemetry, distributed tracing, or operational logs exist**. Software architects and Site Reliability Engineers (SREs) face two fundamental questions without operational data:

1. *Which components, message topics, and communication links are systemically critical to system dependability and performance?*
2. *Why are they critical, and what specific architectural repair (such as replicating a broker, decoupling an over-subscribed topic, or isolating a library) will eliminate that risk?*

These questions also intersect with **computational sustainability** in software engineering [13, 14]. Pre-deployment manifest analysis eliminates the need to provision, maintain, and tear down energy-intensive cloud staging clusters and fault-injection harnesses during verification. However, evaluating pre-deployment tools requires rigorous accounting of developer-side computation [15, 16]. We characterize the wall-clock execution profile across scaling graph topologies, showing that the learned neural forward pass (56 ms) introduces negligible runtime cost (0.02% of pipeline execution), while the dominant computational cost resides in deterministic all-pairs graph feature extraction (Sections 7.5 and 8.2).

1.2. Problem Statement: The Architecture–Code Gap and Black-Box AI

We formulate pre-deployment dependability analysis around two distinct, complementary tasks:

1. **Failure-Impact Forecasting (Predictive Pathway):** Forecasting dynamic cascading failure blast radii and identifying critical components using relation-specific graph learning over topological representations. While closed-form network metrics capture broad connectivity, whether learned relational models resolve multi-hop, typed cascade propagation across heterogeneous topologies is an empirical question evaluated directly in this study (Section 7.1).
2. **Explainable Criticality Attribution (Explanation Layer):** A ranked shortlist reveals *where* risk lies, but not *how to remediate it*. We pair the predictor with an interpretable structural quality profile grounded in ISO/IEC 25010 [17] and ISO/IEC 25019 [18]. This layer diagnoses the *qualitative root cause* of vulnerability—distinguishing single points of failure (Availability) from error-propagation hubs (Fault Tolerance) to guide architectural repairs.

Both pathways operate on the same graph but maintain architectural decoupling: they share no parameters, and neither depends on the other's training representations (Section 4.2). This separation allows the framework to identify components that are structurally central yet operationally low-impact.

Existing techniques fail to bridge what we term the **Architecture–Code Gap**: *a distributed system can have bug-free source code within each individual service, yet harbor catastrophic global outages caused by hidden single points of failure (SPOFs) or mismatched middleware QoS contracts.* Traditional architecture evaluation methods such as ATAM [19, 12] and architectural technical debt assessments [20, 21, 22, 23] rely heavily on manual stakeholder interviews rather than automated quantitative analysis. Conversely, static code analysis (SCA) [24, 25, 26, 27] inspects individual services in isolation and cannot see message queues or cross-host links; chaos engineering [28] requires live running clusters post-deployment; and homogeneous network centrality [29, 30, 31, 32] flattens heterogeneous entities (topics, libraries, brokers, hosts) into untyped graphs.

Furthermore, applying machine learning to system dependability often encounters the **black-box barrier** [33, 34]: deep models output scalar risk scores without actionable engineering rationales. In mission-critical software engineering, developers cannot refactor topologies without understanding *which* architectural mechanism is compromised and *why* a component was flagged.

1.3. The Software-as-a-Graph (SaG) Approach

To bridge the Architecture–Code Gap while overcoming black-box limitations, this work presents **Software-as-a-Graph (SaG)**, an AI-driven pre-deployment **Static System Analysis (SSA)** framework. SaG ingests Architecture-as-Code manifests and executes a four-stage pipeline:

1. **Typed Multigraph Formulation:** SaG models the distributed architecture as a typed, directed multigraph over five core entity types: Applications, Brokers, Topics, Execution Nodes, and Shared Libraries (Section 3.1).
2. **QoS-Aware Logical Dependency Projection:** Using six formal projection rules, SaG derives a semantic `DEPENDS_ON` dependency layer capturing sequential cascades and simultaneous shared-fate blast radii, weighted by declared QoS contracts (Section 3.2).
3. **Heterogeneous Graph Learning (Predictive Pathway):** SaG trains a **Heterogeneous Graph Transformer (HGT)** whose relation-specific attention parameterizes distinct edge types (such as `USES` into libraries vs. `PUBLISHES_TO` into topics). It forecasts cascading blast radii, ranks critical components, and outputs relationship-level criticality (Section 4).
4. **Explainable Quality Attribution (Explanation Layer):** SaG combines code-level metrics with topological properties into a deterministic **Reliability–Maintainability (RM)** attribution model grounded in ISO/IEC 25010 (Section 5), decomposing Reliability into Fault Tolerance and Availability.

To ensure rigorous evaluation, SaG enforces an **input–label independence guarantee**: learned models and attribution baselines operate exclusively on the analytical multigraph G_{analysis}, while ground-truth failure labels are generated by independent discrete-event simulation oracles operating on the raw structural topology $G_{\text{structural}}$ (Section 4.4). Figure 1 illustrates the end-to-end architecture.

Rationale for Graph Learning vs. Direct Simulation. Given that discrete-event simulation $I^*(v)$ supplies ground-truth criticality labels, we examine why machine learning is motivated over running simulation sweeps or closed-form heuristics directly:

- **Inductive Cross-Entity Generalization:** Message passing propagates representations across all entity types simultaneously, allowing the model to predict criticality on unlabelled entities (e.g., brokers, libraries, hosts) where direct fault-injection sweeps are unconfigured.
- **Smooth, Threshold-Marginalized Scoring:** Cascade simulations exhibit stochastic seed variance; a trained neural model learns a smooth surrogate that evaluates architectures rapidly without repeated stochastic simulation iterations.
- **Infrastructure Independence:** Dynamic simulation requires runnable containers and communication harnesses, whereas graph learning evaluates static Architecture-as-Code manifests before runtime infrastructure exists.
- **Relationship-Level Criticality:** Graph neural architectures naturally evaluate edge-level criticalities (I_{edge}) through typed edge projections, supporting channel-level mitigation.

Whether these motivations translate into empirical performance gains over training-free baselines is an empirical question evaluated in Section 7.1.

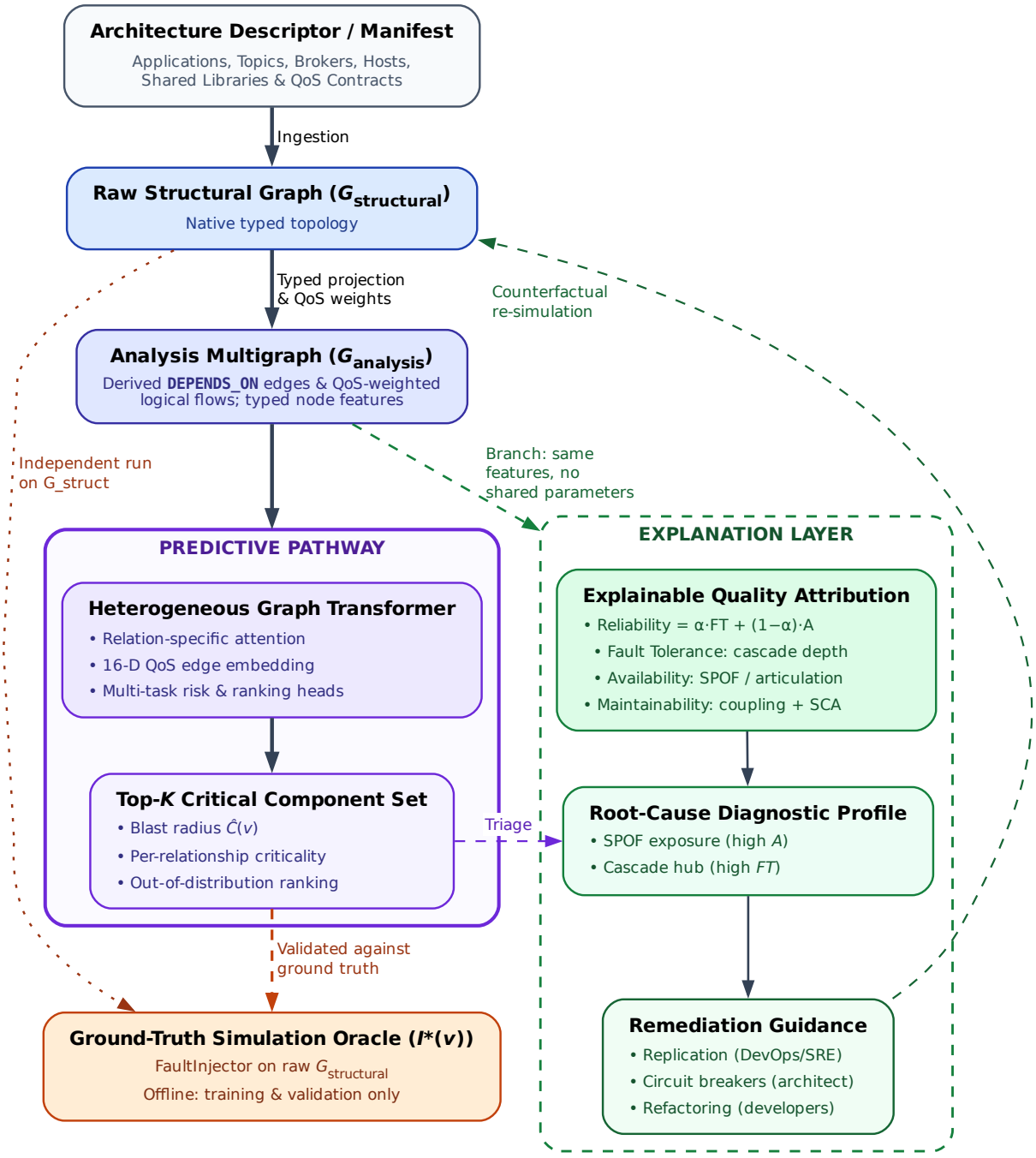

Figure 1: End-to-end architecture of the SaG framework. The predictive pathway (Section 4) processes Architecture-as-Code manifests through typed multigraph construction, QoS-weighted dependency projection, and heterogeneous graph learning to predict component and relationship criticality. Ground-truth discrete-event simulation oracles (Section 4.3) operate offline on $G_{\text{structural}}$ for training and validation. The explanation layer (Section 5) operates on G_{analysis} to produce standards-grounded quality profiles that attribute root causes and guide candidate refactorings verified in the Prescribe stage (Section 5.3).

1.4. Research Questions

This empirical study investigates five research questions:

RQ1 (Predictive Efficacy): *How accurately does heterogeneous graph learning predict cascading failure impact and identify critical components compared with traditional, training-free network metrics?*

RQ2 (Value of Architectural Typing): *Does modeling distinct entity and dependency types yield better failure predictions than homogeneous graph models, and does that advantage hold out-of-distribution on unseen architectures?*

RQ3 (QoS Encoding and Robustness): *Do middleware Quality-of-Service contracts provide predictive signal beyond structural topology, do diverse simulation oracles corroborate one another, and are results robust to hyperparameter variations?*

RQ4 (Real-World Generalization): *How effectively does the framework transfer zero-shot to authentic open-source distributed systems across autonomous vehicles (ROS 2), cloud microservices, IoT meshes, and industrial edge computing?*

RQ5 (Analysis Cost and Computational Sustainability): *What does pre-deployment static analysis cost during CI/CD, which pipeline stage dominates execution time, and how does static analysis compare against simulation oracles?*

1.5. Key Contributions

This paper makes four principal contributions:

1. **Heterogeneous Graph Learning for Pre-Deployment Dependability:** A relation-specific Heterogeneous Graph Transformer (HGT-QoS) forecasting cascading blast radii from Architecture-as-Code manifests, with 16-D continuous-categorical edge features capturing 7 QoS dimensions (Section 4). Under inductive Leave-One-Scenario-Out (LOSO) cross-validation across twelve architectures, typed learning outperforms untyped homogeneous learning by $\Delta\rho = +0.114$ in Spearman correlation (winning 11 of 12 folds, $p = 0.0122$), with QoS encodings contributing an independent $+0.054$ ($p = 0.0093$). Against an unparameterized QoS-weighted centrality baseline, out-of-distribution ranking achieves $+0.127$ ($p = 0.077$), establishing empirical parity boundaries (Section 7.1).
2. **A Formal Typed Multigraph Architecture Model:** A formal representation mapping complex distributed systems into typed multigraphs, deriving logical dependencies from pub-sub interactions and distinguishing sequential starvation cascades from simultaneous shared-fate blast radii (Section 3).
3. **A Standards-Grounded Explanation Layer:** An interpretable Reliability–Maintainability attribution model grounded in ISO/IEC 25010/25019 that decomposes component vulnerability into single-point-of-failure exposure (Availability) and error-propagation reach (Fault Tolerance) to guide concrete architectural remediations (Section 5).
4. **Extensive Empirical Benchmark, Transfer Evaluation, and Cost Profiling:** An empirical study spanning twelve synthetic topologies (2,461 components) and five open-source systems (351 components). We characterize pipeline cost, showing that neural inference represents only 0.02% of runtime (56 ms), while deterministic graph feature extraction dominates (82.7 s vs. 7.2 s for simulation; Sections 6–7).

Relationship to Prior Work. An earlier conference paper [35] introduced the preliminary multigraph formulation and deterministic quality model on synthetic topologies. This JSS manuscript substantially extends that work by: (i) introducing the predictive HGT pathway with 16-D QoS edge encodings and multi-task heads (Section 4); (ii) executing inductive LOSO cross-validation across twelve architectures (Section 7.2); (iii) performing zero-shot transfer across five authentic open-source systems (Section 7.4); (iv) measuring empirical pipeline cost and computational sustainability in CI/CD (Section 7.5); (v) establishing multi-oracle convergent validity across topological, composite, and queue-flow simulation paradigms (Section 4.3); and (vi) conducting global parameter sensitivity screening (Section 7.3).

1.6. Paper Organization

The remainder of this paper is organized as follows: Section 2 reviews related work. Section 3 formalizes the SaG multigraph model and dependency projections. Section 4 details the Heterogeneous Graph Transformer and simulation oracles. Section 5 presents the ISO/IEC-grounded explanation layer. Section 6 describes the experimental methodology. Section 7 reports empirical results for RQ1–RQ5. Section 8 discusses practical implications, threats to validity, and limitations. Section 9 concludes.

2. Related Work

This work builds upon and connects four foundational research areas: (1) dependability, performance, and sustainability in distributed software systems; (2) static code and system analysis; (3) software quality measurement and multi-criteria evaluation; and (4) graph representation learning and explainable AI (XAI).

2.1. Dependability, Performance, and Sustainability in Distributed Software Systems

The publish–subscribe (pub-sub) and asynchronous event-driven paradigms decouple communicating entities in space, time, and synchronization, enabling elastic scalability and high throughput [7]. Modern middleware standards—such as ROS 2 [1], Apache Kafka [2], DDS [3], and MQTT [4]—govern these exchanges through fine-grained Quality-of-Service (QoS) policies that regulate message durability, transport reliability, priorities, and delivery deadlines. In cloud-native microservice meshes and distributed AI/LLM serving backbones, asynchronous message passing and queueing topologies form the primary communication substrate, directly shaping tail latencies, throughput bottlenecks, and hardware resource utilization.

Prior dependability and performance research has focused predominantly on **runtime mechanisms**, including dynamic consensus protocols, broker clustering, adaptive backpressure throttling, autoscaling, and automated failover. In parallel, **chaos engineering and runtime verification** [28] inject faults or latency into staging or production clusters to observe degradation and recovery. While runtime fault injection delivers operational validation that no static method can match, it requires a fully provisioned cluster, carries the risk of real service disruption, and consumes cluster-hours per sweep — which places it, alongside model training, among the development-time computations whose energy cost green software engineering has argued should be accounted for rather than assumed away [13, 15, 16]. In practice, this precludes its continuous execution during architectural design or lightweight commit-level CI/CD.

Our work addresses the complementary **pre-deployment phase**: predicting systemic cascading vulnerabilities and performance degradation directly from Architecture-as-Code descriptors before runtime infrastructure is provisioned. From a sustainability perspective, pre-deployment static analysis fundamentally eliminates the energy and carbon footprint of provisioning, keeping warm, and tearing down live cloud staging clusters. We note that while this provides substantial infrastructure avoidance, static graph feature extraction itself incurs non-trivial computational overhead ($O(|V|^2 + |V||E|)$), which we empirically benchmark against simulation in Sections 7.5 and 8.2.

Architecture-Based Reliability Prediction. Predicting dependability from an architectural description before deployment is an established objective in software engineering. Cheung's absorbing-Markov-chain model [36] derives system reliability from component reliabilities and a transfer-of-control graph; Goseva-Popstojanova and Trivedi [37] systematize the state-based, path-based, and additive families that followed, while Immonen and Niemelä [38] survey architectural reliability estimation. Model-driven descendants such as the Palladio Component Model [39] and layered queueing networks [40] predict performance and reliability from parameterized component specifications.

A parallel tradition explicitly annotates architectural descriptions with fault behaviors or parameters. For example, the AADL Error Model Annex [41] allows architects to declare component error states and propagation paths to generate fault trees and Markov models automatically. Similarly, analytical reliability models require per-component failure probabilities, transition rates, or operational profiles [37, 39]. Where authored, such models answer strictly richer questions than topological rankings. The critical distinction lies in required inputs: analytical and annex-based methods require deliberate, pre-calibrated failure semantics

unavailable at commit time without operational telemetry. SaG addresses a complementary question: given only declared deployment manifests, which components' failures would propagate furthest through the declared topology?

Data-Driven Failure Prediction and Root-Cause Analysis in Microservices. A substantial literature localizes faults in microservice systems from operational data. Seer [42] and Sage [43] predict and debug QoS violations from traces and hardware telemetry; MicroRCA [44] and TraceRCA [45] localize root causes over service-dependency and trace graphs; DeepTraLog [46] and Eadro [47] combine traces, logs, and metrics under graph-based deep models. These approaches achieve high operational precision because they observe the running system. However, they require deployed services actively emitting observability telemetry, precluding their use at design or pull-request time. SaG occupies the pre-deployment complement, operating on static topology rather than dynamic telemetry.

2.2. Static Code Analysis (SCA) vs. Static System Analysis (SSA)

Traditional **Static Code Analysis (SCA)** tools (e.g., SonarQube [24]) inspect source code Abstract Syntax Trees (ASTs) within individual services. They evaluate cyclomatic complexity [25], class cohesion, module coupling (e.g., Lack of Cohesion in Methods [LCOM], Coupling Between Objects [CBO]) [26, 27], and code duplication to flag internal code smells and defect-prone modules [48, 49, 50, 51]. However, SCA cannot observe runtime communication topology: it is blind to inter-service messaging channels, message broker queue saturation, and cross-host failure propagation.

Recovering system-level structure statically is also an active area of research. A body of work reconstructs microservice architecture from source and deployment artifacts without running the system: Bushong et al. [52] derive communication diagrams and bounded contexts from static code analysis of a service mesh, and a recent multivocal review compares nine such recovery tools and finds their outputs complementary [53]. Architecture recovery aims to produce a faithful description of the system as built, typically for comprehension or drift detection. In contrast, SaG takes a declared topology as given and evaluates which of its components a failure would propagate furthest from. Recovery represents an upstream complement that can supply the manifests SaG consumes when Architecture-as-Code descriptions are incomplete.

To bridge this "Architecture–Code Gap," **Static System Analysis (SSA)** extends static analysis from single-service source code to the global system architecture. By modeling distributed applications, message topics, brokers, execution nodes, and shared libraries as a connected multigraph, SSA propagates code-level quality metrics across architectural dependencies. This allows engineering teams to detect structural anti-patterns [54, 23] and architectural technical debt [21] early during continuous integration (CI/CD) [55, 56], before defective topologies enter production.

2.3. Software Quality Models and Multi-Criteria Evaluation

Software product quality is standardized by the **ISO/IEC 25010:2023** product quality model [17] and the **ISO/IEC 25019:2023** Quality-in-Use model [18]. ISO/IEC 25010:2023 defines three closely intertwined characteristics critical to modern distributed systems:

- **Reliability:** The degree to which a system performs specified functions under stated conditions, comprising Faultlessness, Availability, Fault Tolerance, and Recoverability.
- **Maintainability:** The degree of effectiveness and efficiency with which software can be modified, comprising Modularity, Reusability, Analyzability, Modifiability, and Testability.
- **Performance Efficiency:** Performance relative to resource consumption under stated conditions, comprising Time Behavior (latency, response time), Resource Utilization (CPU, memory, bandwidth), and Capacity.

SaG operationalizes a strict subset of these: Availability and Fault Tolerance under Reliability, and Modularity, Modifiability, and Analyzability under Maintainability (Section 5.1). Faultlessness, Recoverability,

Reusability, and Testability are not derivable from deployment topology alone and are outside the scope of this work.

Software engineering measurement explicitly distinguishes between *internal quality* (measured on static artifacts at rest) and *external quality* (measured on executing software systems) [57, 58]. In distributed architectures, architectural debt (such as over-centralized message topics or unreplicated brokers) degrades internal quality and precipitates severe external performance bottlenecks, queue congestion, and outages.

Aggregating multi-attribute structural metrics into an auditable quality score constitutes a classic Multi-Criteria Decision Making (MCDM) problem. The **Analytic Hierarchy Process (AHP)** [59] delivers a structured pairwise-comparison method with an explicit Consistency Ratio ($CR \leq 0.10$) intended to certify that elicited judgments are mutually coherent. In this work, AHP is utilized to establish an audited, explainable Reliability–Maintainability (RM) quality baseline in conjunction with learned graph models (Section 5).

2.4. Graph Representation Learning and Explainable AI

Network science provides established centrality metrics to identify critical nodes, including degree, closeness, betweenness centrality [29, 31], articulation points, and PageRank [30, 32]. Foundational studies on network robustness [10], cascading overloads [8], and interdependent networks [9] model disruption propagation across connected topologies. While percolation models offer natural comparators, our training-free baselines are centrality-based (Section 6.2); evaluating targeted percolation fragmentation remains a recognized future baseline comparison.

However, standard network metrics suffer from two major limitations when applied to software architectures: (1) **Dimensional Collapse**, where a single centrality scalar cannot distinguish *why* a component is critical (e.g., an isolated single point of failure vs. an error-propagating cascade hub vs. an over-shared library); and (2) **Semantic Collapse**, where unweighted metrics treat all nodes and edges identically, conflating fundamentally different architectural entities such as asynchronous message topics, shared libraries, and physical execution hosts.

To overcome hand-engineered metrics, recent studies apply machine learning to network vulnerability (e.g., FINDER [60], DrBC [61], PowerGraph [62]). However, most models rely on **homogeneous message passing** (GCN [63], GraphSAGE [64], GAT [65]), averaging signals indiscriminately across connection types. Because distributed software architectures are inherently **heterogeneous**, homogeneous models blur entity boundaries and fail to generalize out-of-distribution. Heterogeneous Graph Neural Networks (RGCN [66], HAN [67], HGT [68], MAGNN [69]) resolve this via relation-specific transformations. We build upon the **Heterogeneous Graph Transformer (HGT)** [68] to preserve typed relational semantics when forecasting cascade blast radii.

Explainable AI (XAI) vs. The Black-Box Barrier. A critical hurdle in applying modern AI to software engineering is the **black-box barrier**: deep neural models output risk scores or continuous embeddings without explaining underlying structural causality. In production software engineering, uninterpretable risk rankings hinder actionable decision-making: developers and SREs cannot determine whether to replicate a host, configure circuit breakers, or refactor shared libraries.

Existing GNN explanation techniques, such as GNNExplainer [33] and PGExplainer [34], identify influential subgraphs through edge masking or parameterized learning. Although useful, these methods explain the model using internal latent representations rather than standardized software engineering concepts. SaG resolves this limitation through a decoupled dual-pathway design: the predictive HGT pathway reveals typed mutual-attention distributions indicating *which* architectural relations propagated the cascade (Section 7.3.4 and Supplementary Section S8), while the deterministic explanation layer attributes fragility to standardized ISO/IEC quality sub-characteristics (Section 5), translating raw predictions into actionable, cost-effective remediations.

3. The Software-as-a-Graph (SaG) Architectural Model

This section formalizes the Software-as-a-Graph multigraph representation (Section 3.1), the QoS-aware weighting and logical dependency derivation rules (Section 3.2), the dual graph views (Section 3.3), and the typed node feature encodings (Section 3.4).

3.1. Formal Multigraph Definition

A complex distributed software system is formally modeled as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \tag{1}$$

where:

- V is the set of system entities, partitioned into five disjoint entity types:

$$V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}} \tag{2}$$

- E is the set of directed edges connecting entities.
- $\tau_V : V \to \mathcal{T}_V$ and $\tau_E : E \to \mathcal{T}_E$ are typing functions assigning node and edge categories.
- $w_V : V \to [0, 1]$ and $w_E : E \to [0, 1]$ are weighting functions representing entity criticality and connection strength.

Table 1 summarizes the five entity types and six structural edge types formalized in the SaG model, along with their semantics and representative concrete distributed-system implementations.

Table 1: Entity and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	**Architectural Role**	**Concrete System Examples**
Application (V_{app})	Autonomous process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	`/sensor/lidar`, `orders.payment.completed`
Node (V_{node})	Physical host or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	`librdkafka`, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	**Direction**	**Semantic Meaning**
`PUBLISHES_TO`	App/Library → Topic	Component publishes messages to topic
`SUBSCRIBES_TO`	App/Library → Topic	Component consumes messages from topic
`ROUTES`	Broker → Topic	Broker manages and routes topic traffic
`RUNS_ON`	App/Broker → Node	Process is hosted on physical/virtual host
`CONNECTS_TO`	Node → Node	Physical network link between hosts
`USES`	App → Library	Application links to shared library dependency

Application and Library entities additionally incorporate static code metrics computed via Static Code Analysis (SCA) tools (`cm_*` attributes: lines of code, cyclomatic complexity, coupling between objects, LCOM), linking code-level fragility directly to topological analysis.

3.2. QoS-Aware Weights and Logical Dependency Derivation

In distributed middleware, communication links vary in coupling strength based on their Quality-of-Service (QoS) contracts. For instance, a `RELIABLE` topic with `TRANSIENT_LOCAL` durability binds communicating services substantially more tightly than a `BEST_EFFORT` telemetry stream.

Each topic $t \in V_{\text{topic}}$ carries an intrinsic criticality weight $w_V(t) \in [0, 1]$ combining its declared QoS semantics with two runtime-stress modulators: payload size and publication frequency:

$$w_V(t) = \beta \cdot \text{QoS}(t) + \alpha \cdot \text{SizeNorm}(t) + \psi \cdot \text{FreqNorm}(t), \quad (\beta, \alpha, \psi) = (0.75,\ 0.15,\ 0.10) \tag{3}$$

where the QoS term is an AHP-weighted aggregate of the declared contract:

$$\mathrm{QoS}(t) = w_{\mathrm{rel}} \cdot q_{\mathrm{rel}} + w_{\mathrm{dur}} \cdot q_{\mathrm{dur}} + w_{\mathrm{prio}} \cdot q_{\mathrm{prio}}, \quad (w_{\mathrm{rel}}, w_{\mathrm{dur}}, w_{\mathrm{prio}}) = (0.24,\, 0.62,\, 0.14) \tag{4}$$

Here, $q_{\mathrm{rel}}, q_{\mathrm{dur}}, q_{\mathrm{prio}} \in [0, 1]$ represent normalized reliability, durability, and transport-priority scores. Durability dominates because it governs whether data persists across restarts and network partitions. Reliability and transport priority govern in-flight delivery quality, with reliability receiving higher weight because delivery guarantees precede scheduling. The sub-weight vector is the geometric-mean priority vector of an independently stated Saaty pairwise-comparison matrix (Supplementary Section S4; $CR = 0.016$).

The stress modulators are logarithmically compressed and clamped to $[0, 1]$:

$$\mathrm{SizeNorm}(t) = \min\left(1.0,\, \frac{\log_2(1 + \mathrm{bytes})}{20}\right) \tag{5}$$

$$\mathrm{FreqNorm}(t) = \min\left(1.0,\, \frac{\log_{10}(1 + \mathrm{Hz})}{3}\right) \tag{6}$$

The size normalization denominator of 20 reflects a 1 MiB (2^{20} bytes) design envelope, representing the practical DDS sample ceiling before RTPS network fragmentation degrades throughput; the frequency denominator of 3 accommodates up to 1 kHz (10^3 Hz) high-rate sensor streams (e.g., IMU or radar feeds) without saturating dynamic range. The final weight $w_V(t)$ is clamped to $[0.01, 1]$, ensuring best-effort edges remain visible to graph traversals. Every structural communication edge e incident on t (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`) inherits $w_E(e) = w_V(t)$ along with the topic's QoS vector.

The outer split (β, α, ψ) is a declared convex combination. Sweeping it over the full simplex changes the induced ordering of $w_V(t)$ by at most $\rho = 0.081$ and downstream rank correlation by at most 0.031, establishing that it is a documented convention rather than a sensitive tuned parameter (Supplementary Section S1).

3.2.1. Logical Dependency Projection (`DEPENDS_ON`)

Structural edges capture explicit deployment connections but omit implicit runtime dependencies. For example, a subscriber depends upon a publisher, yet no direct edge connects them in pub-sub architectures. We therefore derive a single unified semantic relation, `DEPENDS_ON`, directed from *dependent* to *dependency* ("if target fails, source is impacted"), according to the six projection rules detailed in Table 2:

Table 2: The six `DEPENDS_ON` logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent → Dependency)	Derived Weight ($w_E(e)$)
1	`app_to_app`	Subscriber → Publisher (via shared Topic, incl. transitive `USES`)	$1 - \prod_{t \in T}(1 - w_V(t))$
2	`app_to_broker`	Publisher/Subscriber → Broker routing its topics	$1 - \prod_{t \in T}(1 - w_V(t))$
3	`node_to_node`	Host → Host (lifted from inter-host app dependencies)	Lifted $\max w_E$
4	`node_to_broker`	Host → Broker (lifted from hosted app dependencies)	Lifted $\max w_E$
5	`app_to_lib`	Application → Shared Library it `USES`	$H(w_V(\mathrm{app}), w_V(\mathrm{lib}))$
6	`broker_to_broker`	Broker ↔ Broker (shared physical fault-domain colocation, symmetric)	$w_V(\mathrm{node})$

Rules 1 and 2 aggregate the set of topics T connecting a component pair using a probabilistic union rather than a maximum [70, 71, 72]. This guarantees that additional parallel failure vectors increase coupling monotonically while keeping $w_E \in (0, 1]$. Rule 5 applies the harmonic mean $H(x, y) = 2xy/(x + y)$ [73] to combine the consuming Application's and the shared Library's vertex weights, balancing caller and dependency criticality. Rules 3 and 4 assign the maximum weight among component-level dependencies crossing the host boundary.

3.2.2. Sequential Cascades vs. Simultaneous Blasts

A foundational principle of the SaG model is distinguishing between two fundamentally different degradation modes:

- **Sequential Cascade (Rule 1):** When an application publisher fails, downstream subscribers suffer message starvation. The failure propagates hop by hop through message queues and topic buffers.
- **Simultaneous Blast (Rule 5):** When a shared software library or execution node crashes, all consuming applications and colocated brokers fail *instantaneously* in a single shared-fate event.

Preserving architectural entity types and relation-specific projection rules enables SaG to model both mechanisms, whereas untyped homogeneous graphs collapse them into indistinguishable edges.

Rule 6 is intentionally the only symmetric projection rule. It captures physical fault-domain colocation: when two brokers are colocated on the same host machine, an outage of that host (or hardware resource exhaustion) simultaneously disables both brokers. The derived weight equals the hosting Node's weight ($w_V(\text{node})$). Rule 6 does not model directional application-level broker clustering (e.g., Kafka partition replication or RabbitMQ shovel links), which do not require physical colocation; extending the schema to capture inter-broker application replication is reserved for future work.

3.3. Dual Graph Views and Architectural Layers

The SaG framework maintains two distinct representations of the system:

1. **Structural Graph ($G_{\text{structural}}$):** The raw deployment graph containing physical and structural relations (such as `PUBLISHES_TO`, `ROUTES`, `RUNS_ON`, and `USES`). Discrete-event simulators consume this view exclusively to execute unbiased failure injections (Section 4.3).
2. **Analysis Graph (G_{analysis}):** The projected graph containing derived `DEPENDS_ON` edges annotated with QoS weights and ingested SCA code metrics. All GNN feature representations, graph embeddings, and analytical metrics are computed on G_{analysis}.

Figure 2 illustrates this duality on a running example, contrasting the raw structural graph against the derived `DEPENDS_ON` projection.

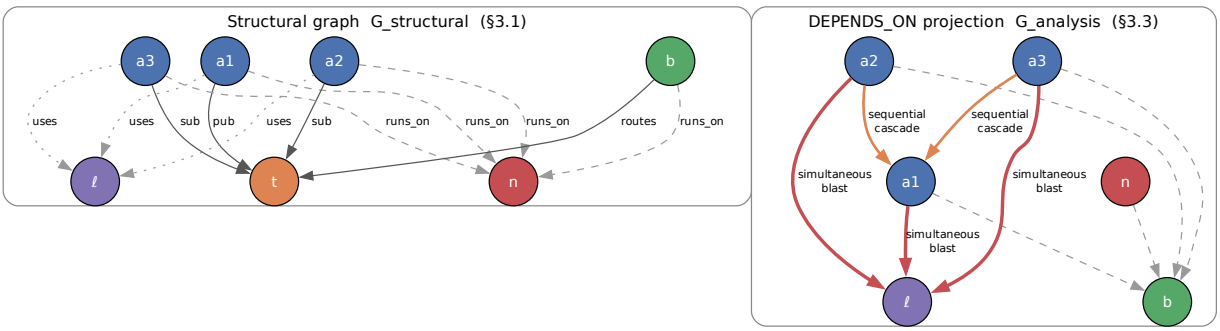

Figure 2: Running example: raw structural graph (left) and the derived `DEPENDS_ON` projection (right). The projection derives implicit runtime dependencies (subscribers depending on publishers) while independent simulation oracles operate exclusively on the structural view.

G_{analysis} is further structured into four analytical layers (Application, Middleware, Infrastructure, and Global System), enabling evaluation of criticality at subsystem levels, consistent with hierarchical frameworks such as MIL-STD-498 [74].

3.4. Typed Node Feature Encoding

Both pathways read the same typed node properties from G_{analysis}: the predictive pathway (Section 4) projects them per entity type before heterogeneous message passing, and the explanation layer (Section 5) aggregates them into its quality profile. All five entity types share indices 0–17, a common block of topological metrics (in/out degree, betweenness, closeness, reverse PageRank, clustering coefficient, articulation score, bridge load) produced by the deterministic analysis stage whose cost is characterized in Section 7.5. Type-specific features extend that block:

- **Application (23 dims):** indices 18–22 add source-code metrics from SCA — lines of code, cyclomatic complexity, Martin's instability $I_{\text{code}} = C_e/(C_a + C_e)$ [75], Lack of Cohesion in Methods, and the composite Code Quality Penalty (CQP).
- **Library (25 dims):** the Application block plus two library-specific blast-radius drivers (23–24): the normalized size of the transitive reverse-`USES` closure, and the normalized count of distinct subscribers reachable from topics published within that closure.
- **Broker (19 dims):** index 18 is normalized queue buffer capacity.
- **Topic (22 dims):** indices 18–21 are publisher count, subscriber count, log message frequency $\log(1 + \text{freq})$, and ordinal QoS criticality.
- **Infrastructure Node (20 dims):** indices 18–19 are normalized CPU core allocation and physical memory.

4. Graph Learning for Failure-Impact Prediction

Cascading failure impact in complex distributed software systems is inherently non-linear, multi-hop, and relation-dependent. Outages propagate not merely based on immediate neighbor connectivity, but through heterogeneous architectural relations and middleware contracts extending multiple hops beyond the initial fault. To model these compound dynamics, the predictive pathway of SaG employs a relation-specific graph neural network evaluated against both training-free topological heuristics and ground-truth simulation oracles.

This section details the Heterogeneous Graph Transformer (HGT) architecture and its typed edge encodings (Section 4.1), the multi-task prediction heads and dimension-masked loss formulation (Section 4.2), the ground-truth simulation oracles (Section 4.3), and the input–label independence guarantee that prevents data leakage (Section 4.4).

4.1. Heterogeneous Graph Transformer Architecture

Because distributed systems comprise heterogeneous entity types (Applications, Libraries, Brokers, Topics, Infrastructure Nodes) and diverse interaction semantics (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`, `DEPENDS_ON`), we employ a three-layer **Heterogeneous Graph Transformer (HGT)** architecture [68], implemented within PyTorch Geometric [76], with hidden dimension $D = 64$ and $H = 4$ attention heads. This architecture ensures that typed relations, rather than simple adjacency, govern failure-impact forecasting.

4.1.1. Continuous-Categorical Edge Feature Encoding (16-D)

To capture continuous QoS constraints and channel semantics, SaG encodes each directed edge $e = (u, v)$ as a 16-dimensional continuous-categorical vector $e_{uv} \in \mathbb{R}^{16}$. The representation comprises 15 active operational dimensions and one reserved extension dimension:

- **Topological and Relational Encodings (Indices 0–8):** Index 0 is the scalar coupling weight $w_E(e) \in (0, 1]$ of Section 3.2; index 1 is the normalized count of simple paths traversing e in G_{analysis}; indices 2–8 provide a one-hot encoding across the seven structural and derived relations (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`, `DEPENDS_ON`).
- **Active Middleware QoS Dimensions (Indices 9–14):** Six active dimensions encode declared middleware QoS parameters on communication edges (zeroed on non-messaging edges): reliability (0 best-effort, 1 reliable); durability (0 volatile, 0.5 transient-local, 0.6 transient, 1 persistent); message priority (0, 0.33, 0.66, 1); a heterogeneity indicator flagging when an edge departs from its scenario's modal profile; and the delivery deadline pair (a boolean active flag and $\log_{10}(1 + \text{deadline\_ns}/10^6)$), where deadlines are populated across 75% of topics (463 of 615) in our evaluation corpus.

- **Reserved Real-Time Extension (Index 15):** Index 15 ($\log_{10}(1 + \text{max\_blocking\_ms})$) is an architectural schema provision reserved for hard real-time microsecond profiles; it is uniformly zero throughout the current benchmarks.

An edge projection module maps e_{uv} into the hidden space: $e'_{uv} = W_{\text{edge}} e_{uv}$. Prior to relational attention computation, this projection vector is incorporated directly into the target node representation: $\tilde{h}_v = h_v + e'_{uv}$.

4.1.2. Type-Specific Projection and Heterogeneous Message Passing

For each source node u and target node v connected by meta-relation $\tau(e) = (\tau(u), \phi(e), \tau(v))$:

1. **Type-Specific Projection:** Node feature vectors x_v (of dimension 19–25 depending on entity type $\tau(v)$) are mapped into the shared D-dimensional hidden space:

$$h_v^{(0)} = \text{LayerNorm}\big(\text{GELU}(W_{\tau(v)} x_v)\big) \tag{7}$$

2. **Relational Mutual Attention:** Type-parameterized Query (Q), Key (K), and Value (V) projections calculate relation-specific attention. For head $i \in \{1, \ldots, H\}$, with the softmax taken over the incoming neighborhood $\mathcal{N}(v)$:

$$\text{Attn}^{\,i}(u, e, v) = \underset{u \in \mathcal{N}(v)}{\text{Softmax}} \left(K^i(u)\, W^i_{\text{att},\phi(e)}\, Q^i(\tilde{h}_v)^\top \cdot \frac{\mu_{\langle \tau(u), \phi(e), \tau(v) \rangle}}{\sqrt{D/H}} \right) \tag{8}$$

where $\mu_{\langle \tau(u), \phi(e), \tau(v) \rangle}$ is the learned per-meta-relation scaling prior [68], which weights entire relation triples up or down independently of individual node embeddings. Message passing is parameterized by:

$$\text{Msg}(u, e, v) = V(u) W_{\text{msg},\phi(e)} \tag{9}$$

3. **Bidirectional Message Passing:** To capture downstream consumer starvation and upstream broker backpressure simultaneously, message passing executes across both forward and transposed graph views (G_{analysis} and $G^\top_{\text{analysis}}$).
4. **Residual Aggregation and Layer Normalization:** Target representations update across layers $l \in \{1, \ldots, L\}$ via residual connections and layer normalization:

$$h_v^{(l)} = \text{LayerNorm}\left(h_v^{(l-1)} + \text{Dropout}\left(\sum_{u \in \mathcal{N}(v)} \text{Attn}(u, e, v) \cdot \text{Msg}(u, e, v) \right) \right) \tag{10}$$

Training Protocol and Optimization Hyperparameters. Models are optimized using AdamW with initial learning rate $\eta = 3 \times 10^{-4}$, weight decay 10^{-4}, and dropout probability $p = 0.10$ applied post-attention. Learning rates follow a cosine annealing schedule with warm restarts ($T_0 = 75$, $T_{\text{mult}} = 2$, $\eta_{\min} = 3 \times 10^{-6}$). Training runs for at most 300 epochs with early stopping governed by a patience of 30 epochs monitored on validation loss. Inductive subgraphs are processed per scenario using full-graph inductive packing without mini-batch subsampling, with validation masks isolating held-out nodes. Five independent random seeds $\{42, 123, 456, 789, 2024\}$ are evaluated across all runs, redrawing partition masks and initializations.

4.2. Multi-Task Prediction Heads and Dimension Masking

From the final node embeddings $h_v^{(L)}$, SaG utilizes specialized multi-task prediction heads:

- **Reliability Head:** $\hat{R}(v) = \sigma(\text{MLP}_R(h_v)) \in [0, 1]$
- **Maintainability Head:** $\hat{M}(v) = \sigma(\text{MLP}_M(h_v)) \in [0, 1]$
- **Composite Failure Impact Head:** $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{R}(v) \parallel \hat{M}(v))) \in [0, 1]$
- **Relationship Criticality Head:** $\hat{Q}(u, v) = \sigma(\text{TypedEdgeEncoder}_{\phi(e)}(h_u, h_v, e_{uv})) \in [0, 1]$

4.2.1. Dimension-Masked Loss Formulation

The combined optimization objective integrates regression accuracy, multi-task dimension learning, ranking fidelity, pairwise ordering, and edge prediction:

$$\mathcal{L} = \mathcal{L}_{\text{composite}} + 0.5 \cdot \mathcal{L}_{\text{dimension}} + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} + 0.3 \cdot \mathcal{L}_{\text{edge}} + \lambda_{\text{RM}} \cdot \mathcal{L}_{\text{consistency}} \tag{11}$$

where $I^*(v)$ is simulated cascade impact from the primary oracle (Section 4.3), $\mathcal{L}_{\text{composite}} = \text{MSE}(\hat{I}^*(v), I^*(v))$, and $\mathcal{L}_{\text{rank}}$ is the ListMLE listwise ranking loss [77]:

$$\mathcal{L}_{\text{rank}} = -\frac{1}{N} \sum_{i=1}^{N} \left(\frac{\hat{s}_{\pi_i}}{\tau} - \log \sum_{j=i}^{N} \exp\left(\frac{\hat{s}_{\pi_j}}{\tau} \right) \right) \tag{12}$$

where π denotes the permutation of nodes sorted in descending order of ground-truth impact $I^*(v)$, and $\hat{s}_v = \hat{I}^*(v)$ with baseline temperature $\tau = 1.0$. Pairwise ordering is guided by margin-ranking loss $\mathcal{L}_{\text{pairwise}} = \frac{1}{|P|} \sum_{(u,v)\in P} \max(0, \gamma - (\hat{s}_u - \hat{s}_v))$ with margin $\gamma = 0.05$ over pairs $P = \{(u, v) \mid I^*(u) - I^*(v) > \gamma\}$, and $\mathcal{L}_{\text{consistency}} = \text{MSE}([\hat{R}(v), \hat{M}(v)]_{v\in\text{unlabeled}}, [R_{\text{RM}}(v), M_{\text{RM}}(v)]_{v\in\text{unlabeled}})$ regresses predicted heads toward the diagnostic baseline (Section 5).

Pathway Decoupling Guarantee. Headline models set $\lambda_{\text{RM}} = 0$, guaranteeing that the learned predictive pathway and the deterministic explanation layer remain strictly independent and share no training gradients. **Dimension Masking:** Because dynamic cascade simulation ($I^*(v)$ via `FaultInjector`) observes runtime failure reachability rather than source-code maintainability, maintainability ground truth is unobserved during dynamic simulation. A separate change-propagation oracle $I_M(v)$ evaluates static structural change ripple at the Validate stage, but is withheld from training to prevent circular supervision. We enforce a boolean dimension mask $m = [m_R, m_M] = [1, 0]$:

$$\mathcal{L}_{\text{dimension}} = \frac{1}{\sum_d m_d} \sum_{d\in\{R,M\}} m_d \cdot \text{MSE}(\hat{d}(v), d^*(v)) \tag{13}$$

ensuring that the unobserved maintainability head receives no artificial penalty. The supervised reliability target aligns with systemic failure reachability: $R^*(v) = I^*(v)$, regressing $\hat{R}$ toward overall cascade impact while training distinct projection heads.

4.2.2. Domain-Reweighted Criticality (Q_{domain})

To ground predictions in ISO/IEC 25019 Context of Use ($\vec{\omega} = [q_R, q_M]^\top$), the composite score can be evaluated as:

$$Q_{\text{domain}}(v) = q_R \cdot \hat{R}(v) + q_M \cdot M_{\text{static}}(v) \tag{14}$$

combining learned dynamic reliability with static source-code maintainability.

4.3. Ground-Truth Simulation Oracles

To evaluate predictive accuracy prior to deployment without relying on production runtime telemetry, SaG executes discrete-event failure simulations over the raw structural multigraph $G_{\text{structural}}$. We establish a formal taxonomy of four component-level oracles and one relationship-level oracle:

- **Cascade Reachability Oracle ($I^*(v)$)**, via `FaultInjector`: crashes component v, propagates outages across dependent topics, brokers, and links by breadth-first traversal, and returns the mean fractional feed loss over the subscriber population. A topic's feed loss is the fraction of its publishers that have failed (or failed routers for un-published topics), scaled by a QoS ladder ($\times 1.2$ `RELIABLE`, $\times 1.15$ high/urgent priority, $\times 1.05$ medium) and clamped to $[0, 1]$. This is the **primary continuous target label** throughout.

- **Multi-Metric Composite Oracle ($I_{\mathbf{comp}}(v)$)**, via `FailureSimulator`:

$$I_{\text{comp}}(v) = 0.35 \cdot \Delta\text{Reachability} + 0.25 \cdot \Delta\text{Fragmentation} + 0.25 \cdot \Delta\text{Throughput} + 0.15 \cdot \Delta\text{FlowDisruption} \tag{15}$$

  weighted by operational severity $s(t) = w_V(t) \cdot \text{rate}(t)$. Coefficients derive from Saaty pairwise comparisons with shrinkage (Supplementary Section S4).

- **Dynamic Queue-Flow Oracle ($I_{\mathbf{dyn}}(v)$)**, via `MessageFlowSimulator` on SimPy [78]: simulates message emission rates, stochastic latencies, broker buffer saturation, and queue drops under fault injection, measuring the delivered message drop to surviving consumers.

- **Change-Propagation Oracle ($I_M(v)$)**, via `ChangePropagationSimulator`: a deterministic reverse-dependency traversal quantifying maintenance change impact:

$$I_M(v) = 0.45\,\text{ChangeReach}(v) + 0.35\,\text{WeightedChangeImpact}(v) + 0.20\,\text{NormalizedChangeDepth}(v) \tag{16}$$

- **Relationship Removal Oracle ($I_{\mathbf{edge}}(u, v)$):** the systemic impact of severing one dependency while both endpoints remain operational:

$$I_{\text{edge}}(u, v) = \bar{I}_{\text{comp}}\big(G \setminus \{(u, v)\}\big) - \bar{I}_{\text{comp}}(G) \tag{17}$$

Primary Oracle Role Assignment. Because the three reliability-facing oracles measure distinct operational constructs, we designate $I^*(v)$ **as the primary oracle** for predictive ranking (Tables 6–8). $I_{\text{comp}}(v)$ is reserved for quality gating and prescriptive verification, $I_{\text{dyn}}(v)$ serves as an independent convergent-validity probe (Section 7.3.2), and $I_M(v)$ serves as a structural maintainability reference.

Cross-Oracle Convergent Validity. Measured across the twelve inductive folds on the Application population, the mean Spearman rank correlation is $\rho = 0.620$ for (I_{dyn}, I^*), $\rho = 0.395$ for (I_{comp}, I^*), and $\rho = 0.366$ for $(I_{\text{comp}}, I_{\text{dyn}})$ (Table 10). The agreement between the behavioral queue-flow simulator and the topological cascade injector provides independent convergent evidence across simulation paradigms without collapsing into redundant measurements.

4.4. Input–Label Independence Guarantee

To eliminate data leakage and ensure rigorous evaluation, SaG enforces strict architectural separation between inputs and labels:

- **Feature Space:** Constructed exclusively from G_{analysis} using static structural topology, static code analysis (SCA) metrics, and declared QoS contracts.

- **Label Space:** Evaluated exclusively on raw $G_{\text{structural}}$ through independent simulation oracles (`FaultInjector`, `FailureSimulator`, `MessageFlowSimulator`).

No simulation outputs, failure traces, or dynamic execution telemetry are ever exposed as input features to the GNN or the explanation layer.

Construct Validity and Rationale for Deep Learning. Because G_{analysis} is a projection of $G_{\text{structural}}$, ground-truth labels are a functional of the underlying graph topology. In this context, graph learning functions as an inductive surrogate that learns non-linear cascade propagation rules across heterogeneous schemas. Although direct BFS simulation executes rapidly on raw graphs (7.2 s on 520 nodes; Section 7.5), the deep learning model provides distinct capabilities: (i) scoring unlabelled architectural entity types (brokers, libraries, nodes) where fault injection is unconfigured; (ii) predicting continuous edge criticalities (I_{edge}); and (iii) delivering sub-second forward inference (56 ms) without requiring containerized execution environments.

5. The Explanation Layer: Standards-Grounded Criticality Attribution

The predictive pathway of Section 4 forecasts *where* systemic risk lies. However, scalar ranking alone does not indicate *how to remediate* vulnerability. A component may be critical because it represents an unreplicated single point of failure, because it propagates error cascades widely, or because it exhibits severe coupling debt. These distinct failure modes necessitate different engineering remediations: replicating a broker, inserting circuit breakers, or refactoring module dependencies.

To provide actionable root-cause diagnoses, SaG pairs its predictor with a deterministic, standards-grounded quality profile. Operating over the typed node features (Section 3.4) on G_{analysis}, this explanation layer shares no parameters with the neural model and functions as an independent triage mechanism (Figure 1).

5.1. Grounding in ISO/IEC Standards

In accordance with **ISO/IEC 25010:2023** (Product Quality Model) [17], **ISO/IEC 25019:2023** (Quality-in-Use) [18], and **ISO/IEC 25022:2016** (Measurement of Quality-in-Use) [79], SaG formalizes two primary criticality constructs:

- **Component Criticality (D_1):** The degree to which the failure or severe degradation of an individual component reduces the system's ability to deliver required functionality within its operational context of use.
- **Relationship Criticality (D_2):** The degree of systemic service degradation resulting from the severance or failure of a specific communication channel while both endpoint components remain operational.

Criticality is evaluated across two orthogonal quality characteristics: **Reliability (R)** and **Maintainability (M)**. The two dimensions are separated because they imply distinct architectural remedies. Table 3 outlines this Reliability–Maintainability (RM) quality decomposition, mapping ISO/IEC sub-characteristics to architectural diagnostics, underlying graph metrics, and targeted remediations.

Table 3: The Reliability–Maintainability (RM) quality decomposition.

Dimension	Sub-Characteristic	Architectural Question	Underlying Graph Metrics	Role / Remediation
Reliability (R)	**Fault Tolerance** (FT)	How broadly does failure propagate?	Reverse PageRank on $G^\top$, in-degree, cascade depth	Reliability Eng.: add redundancy, circuit breakers
	Availability (A)	Is this a single point of failure?	Directed articulation score (raw + QoS-weighted), bridge ratio, CDI	DevOps/SRE: replicate host/broker
Maintainability (M)	**Modularity/Modifiability**	How complex and coupled is this?	Betweenness, QoS-weighted out-degree, Code Penalty, coupling imbalance	Architect: refactor code, decouple topics

Coverage Scope: SaG focuses specifically on Reliability and Maintainability. Safety (which requires domain-specific hazard logs, such as ISO 26262 ASIL ratings) and Security (which requires threat models, such as STRIDE) fall outside structural topology analysis and represent domain-specific extensions.

5.2. Composite Quality Score Formulation

All raw topological and code metrics are rank-normalized to $[0, 1]$. Quality sub-characteristics are formulated hierarchically using the Analytic Hierarchy Process (AHP) [59]:

1. **Fault Tolerance ($FT(v)$):** Measures error cascade potential on the transpose graph $G^\top_{\text{analysis}}$ (where edges follow failure propagation from dependency to dependent):

$$FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v) \tag{18}$$

   where $\text{RPR}(v)$ is Reverse PageRank, $\text{Deg}_{\text{in}}(v)$ is normalized in-degree, and $\text{CDPot}_{\text{enh}}(v)$ is the enhanced Cascade Depth Potential.

2. **Availability ($A(v)$):** Identifies structural single points of failure (SPOFs) across five terms:

$$A(v) = 0.2563 \cdot \text{AP}_c^{\text{dir}}(v) + 0.1998 \cdot \text{QSPOF}(v) + 0.1998 \cdot \text{BR}(v) + 0.2563 \cdot \text{CDI}(v) + 0.0878 \cdot w_V(v) \tag{19}$$

   where $\text{AP}_c^{\text{dir}}(v)$ is Directed Articulation Point severity, $\text{QSPOF}(v)$ is QoS-weighted Single Point of Failure severity, $\text{BR}(v)$ is Bridge Ratio (edge-level irrecoverability), $\text{CDI}(v)$ is Connectivity Degradation Index, and $w_V(v)$ is the component's intrinsic weight.

3. **Reliability ($R(v)$):** Blends Fault Tolerance and Availability:

$$R(v) = r_\alpha \cdot FT(v) + (1 - r_\alpha) \cdot A(v), \quad r_\alpha = 0.36 \tag{20}$$

The blend weight $r_\alpha = 0.36$ is screened in Supplementary Section S1. The intra-dimension weights apply $\lambda = 0.70$ shrinkage blending with a uniform prior (Section 7.3). Because three of the comparison matrices are rank-one by construction (implying near-zero CR by design; Supplementary Section S4), these weights represent a structured multi-criteria baseline convention designed for qualitative attribution rather than scalar ranking optimization.

4. **Maintainability ($M(v)$):** Evaluates structural coupling combined with code-level static analysis:

$$M(v) = 0.35 \cdot \mathrm{BT}(v) + 0.30 \cdot w_{\mathrm{out}}(v) + 0.15 \cdot \mathrm{CQP}(v) + 0.12 \cdot \mathrm{CouplingRisk}_{\mathrm{enh}}(v) + 0.08 \cdot (1 - \mathrm{CC}(v)) \tag{21}$$

where $\mathrm{BT}(v)$ is Betweenness Centrality, $w_{\mathrm{out}}(v)$ is QoS-weighted efferent coupling, $\mathrm{CQP}(v)$ is Code Quality Penalty, $\mathrm{CouplingRisk}_{\mathrm{enh}}(v)$ is coupling imbalance, and $\mathrm{CC}(v)$ is local Clustering Coefficient.

The baseline composite quality score $Q(v)$ combines both dimensions:

$$Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v) \tag{22}$$

Components are categorized into adaptive criticality tiers using Tukey quartile thresholds: **CRITICAL** ($Q > Q_3 + 1.5 \cdot \mathrm{IQR}$), **HIGH** ($Q_3 < Q \leq Q_3 + 1.5 \cdot \mathrm{IQR}$), **MEDIUM** ($Q_1 < Q \leq Q_3$), and **MINIMAL** ($Q \leq Q_1$). This enables targeted remediation: high A with low FT indicates an isolated SPOF calling for broker or host replication, whereas high FT denotes an error-cascade hub requiring circuit breakers, rate limiting, and bulkhead isolation.

5.3. Prescriptive Remediation and Counterfactual Verification

Downstream in the pipeline, once the explanation layer attributes an architectural root cause, automated refactoring operators generate candidate repair manifests (e.g., replicating an SPOF broker, inserting circuit breakers, or decoupling publisher–subscriber feeds).

To verify proposed interventions prior to deployment without risking regressions, SaG incorporates a closed-loop **Prescribe Stage** powered by the `EditVerifier`:

- **Counterfactual Mutation Verification:** The verifier constructs an in-memory mutated graph G' reflecting the candidate refactoring and re-executes multi-threshold cascade simulations. An edit is accepted if and only if it strictly reduces systemic composite impact beyond simulator seed variance ($\Delta \bar{I}_{\mathrm{comp}} > \kappa \cdot \sigma_{\mathrm{seed}}$, with $\kappa \geq 1.0$) across every propagation threshold, while introducing zero new articulation points.

- **Relationship-Level Interventions:** For edge-level refactorings (e.g., splitting a congested topic or severing an unstable dependency), the edge removal oracle ($I_{\mathrm{edge}}(u, v)$, Eq. (17)) verifies whether candidate edge modifications mitigate systemic vulnerability.

6. Experimental Setup

6.1. Datasets and System Corpus

The evaluation corpus comprises 2,812 components across seventeen system architectures — twelve synthetic topologies that form the inductive cross-validation folds, and five real-world reference systems withheld entirely from training — as detailed in Table 4:

Here, $|V|$ is the sum of all five entity-type counts per scenario. The twelve synthetic topologies total 2,461 components; the eleven carrying the manifest's `evaluation` role account for 2,387 of these and the ATM case study for the remaining 74. The five real-world systems add 351. $|E|$ counts every raw structural relationship instance recorded in the scenario specification — the native substrate that simulation oracles traverse — rather than the derived `DEPENDS_ON` projection constructed for GNN training. All six structural relation

Table 4: Experimental evaluation corpus across seventeen distributed architectures. The twelve synthetic topologies form the inductive Leave-One-Scenario-Out folds of Table 8; the five real-world systems are withheld from training and used strictly for zero-shot transfer (Section 7.4). Structural edge counts $|E|$ enumerate all physical and logical relation instances in the scenario specification, including `CONNECTS_TO` physical host links, which are omitted from generator summary logs.

Dataset / Architecture	System Paradigm	$\|V\|$	$\|V_{\text{app}}\|$	Topics	Brokers	Hosts	Libs	$\|E\|$
Synthetic evaluation scenarios (`evaluation` role in the manifest)								
Autonomous Vehicle (AV)	ROS 2 Cyber-Physical	152	80	40	4	8	20	774
Enterprise Pub-Sub	Kafka Event Mesh	520	300	120	10	40	50	3,216
Financial Trading	Low-Latency Pub-Sub	124	60	35	5	6	18	631
Healthcare Integration	HL7/FHIR Event Mesh	98	50	25	3	8	12	389
Hub-and-Spoke Enterprise	Broker-Centric Messaging	139	70	30	2	12	25	691
IoT Smart City	MQTT Telemetry Mesh	326	200	80	6	30	10	1,188
Microservices Mesh	Cloud-Native Services	186	90	45	6	15	30	678
Telecom RAN	5G Radio Access Network	225	120	55	8	20	22	881
Industrial SCADA	Plant Control Telemetry	254	140	70	4	25	15	824
Real-Time Gaming	Multiplayer State Sync	158	75	38	5	12	28	630
Logistics Fleet	Vehicle Telematics Mesh	205	110	50	7	18	20	755
Synthetic case study (LOSO fold; `case_study` role in the manifest)								
Air Traffic Management (ATM)	ICAO Global ATM Concept	74	26	27	5	8	8	261
Real-world reference systems (never used as training folds)								
Autoware.universe [80]	Real-World ROS 2 Autoware	75	32	24	3	6	10	179
Cloud Microservices [81]	Real-World GCP Boutique	60	22	20	4	6	8	128
Train-Ticket [82]	Real-World Microservices	90	41	30	3	8	8	162
Home Assistant [83]	Real-World Smart Home	63	24	22	3	6	8	119
EdgeX Foundry [84]	Real-World Industrial IoT	63	22	24	3	6	8	112
Synthetic subtotal (12 LOSO folds)		**2,461**	1,321	615	65	202	258	**10,918**
Real-world subtotal (5 systems)		**351**	141	120	16	32	42	**700**
Total		**2,812**						**11,618**

types are included in that count (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`). Every count in Table 4 is read directly from committed topology files asserting byte-identical regeneration in CI/CD (Section 6.1.1).

Five real-world architectures were transcribed from authentic open-source repositories using dedicated architectural adapters. The synthetic scenarios were produced by a parameterized topology generator: each is fully defined by a committed configuration specifying a random seed, per-entity-type counts, seven-number summaries for application publish and subscribe fan-out, applications per host, library fan-in, topic payload size, and categorical distributions over the three QoS dimensions. Graph degree distributions and clustering emerge directly from these parameters rather than being synthetically forced. Supplementary Section S5 reports the generative parameters governing each topology's shape.

Corpus Composition across Experimental Regimes. The experimental evaluation spans three complementary regimes with precisely bounded corpora:

1. *In-Distribution Evaluation (Table 6):* Evaluated across all twelve distributed architecture scenarios ($n = 12$) from Table 4 using stratified 60% train / 20% validation / 20% test node splits over five random seeds.
2. *Inductive Leave-One-Scenario-Out (LOSO) Cross-Validation (Table 8):* Evaluated across twelve distinct inductive folds totaling 2,461 components. In each fold, models are trained on eleven graphs and tested zero-shot on the held-out twelfth graph.
3. *Real-World Architectural Transfer (Table 11; Supplementary Section S7):* The five open-source real-world systems are withheld entirely and used strictly for zero-shot architectural transfer validation.

Table 5 delineates the exact corpus subset backing each analysis.

6.1.1. Reproducibility of the Corpus

The benchmark corpus is designed to be fully regenerable: each dataset deterministically regenerates from its configuration file via `python cli/generate_graph.py batch -input-dir data/scenarios -output-dir <dir>`. A companion manifest records the random seed, entity counts, git commit hash, and a

Table 5: Corpus subset backing each analysis.

Analysis	**Scenario subset**	n	**Reported in**
In-distribution ranking	Eleven evaluation scenarios + ATM	12	Tables 6–7
Inductive LOSO	Eleven evaluation scenarios + ATM	12	Table 8, Sections 7.1–7.2
QoS edge-feature ablation	Same twelve LOSO folds	12	Section 7.3.1
Weight sweeps and Morris screening	Six–seven core synthetic domains	6–7	Supplementary S1
Cross-oracle convergent validity	Same twelve LOSO folds	12	Table 10
Anti-pattern detection, stratification	Seven core domains + ATM	8	Section 7.3.3
Relational attention illustration	ATM case study alone	1	Supplementary S8
Real-world zero-shot transfer	Five open-source systems	5	Table 11, Supp. S7

SHA-256 cryptographic digest for each emitted topology. Continuous integration regression tests assert that every committed dataset regenerates byte-identically and that all disk digests match the manifest.

6.2. Baselines and Evaluated Predictors

We evaluate four primary predictor configurations drawn from three families. Predictor names state the family and substrate: an `-N` suffix marks a model trained on the *native* multigraph, its absence the derived Application–Library flow projection, and a `-QoS` suffix marks a configuration that consumes declared QoS contracts:

1. **Heterogeneous graph learning (typed HGT). HGT-QoS** (proposed): relation-specific Heterogeneous Graph Transformer (Section 4) ingesting the complete native multigraph with 16-dimensional continuous-categorical edge features that encode middleware QoS contracts. Its ablation **HGT**, which masks those QoS dimensions, is reported in Section 7.3.1.
2. **Homogeneous graph learning (untyped GAT). GAT-N-QoS**: homogeneous Graph Attention Network [65] trained on the identical native multigraph substrate with per-type input projections, but untyped, single-relation message passing. Its edge channel carries scalar QoS coupling $w_E(e)$. The corresponding unweighted ablation is **GAT-N**.
3. **QoS-weighted structural baseline (training-free). Topo-QoS**: QoS-weighted topological centrality evaluated on the derived application flow projection.
4. **Unweighted structural baseline (training-free). Topo**: structural centrality combining unweighted betweenness centrality and articulation point scoring on the flow projection.

In addition, the out-of-distribution evaluation (Table 8) reports **RM** ($Q(v)$, the deterministic quality attribution model of Section 5) as a diagnostic reference baseline.

6.2.1. Evaluation Substrates and Parity Guarantee

Predictors operate on matched substrates within their respective families:

- **Graph Learning Models (GAT-N-QoS, HGT-QoS):** Under Leave-One-Scenario-Out (Table 8), both learned predictors ingest the complete native typed multigraph across all five entity types. Node type reaches homogeneous GAT-N-QoS through its per-type input projection layer, while message passing uses untyped GATConv with shared weights across edges; heterogeneous HGT-QoS employs relation-specific HGTConv weight matrices per edge triple alongside edge-type encodings.

- **Methodological Controls and Parameter Matching:** On the primary training graph, default HGT-QoS carries 434,620 parameters compared to GAT-N-QoS's 28,168 parameters, and incorporates bidirectional message passing. To eliminate capacity and directionality confounds, Section 7.2 introduces three explicit control arms: (i) a parameter-matched homogeneous baseline (439,272 parameters); (ii) a bidirectional homogeneous baseline (429,992 parameters); and (iii) a forward-only heterogeneous model (330,895 parameters).

- **Training-Free Structural Baselines (Topo, Topo-QoS):** Evaluated on the derived Application–Library `DEPENDS_ON` projection (Section 3.2). This projected substrate is necessary because in raw pub-sub multigraphs, Application nodes do not route messages directly, yielding near-zero betweenness on raw structural graphs.

- **Ground-Truth Independence Guarantee:** Crucially, **no predictor in either group accesses** $G_{\textbf{structural}}$: that raw topology is strictly reserved for simulation oracles (Section 4.4), formally asserted by `tests/test_independence_guarantee.py`.

6.3. Evaluation Metrics and Protocols

Figure Accessibility. All figures utilize colorblind-safe Okabe–Ito palettes combined with distinctive markers, hatching, and shapes, ensuring full monochrome legibility.

- **Ranking Precision:** Evaluated via Spearman rank correlation (ρ) and Kendall's rank correlation (τ) between predicted rankings and simulated cascade impact $I^*(v)$ from the primary oracle (Section 4.3).

- **Critical-Set Identification:** Measured via top-K critical-set overlap (Overlap@K, reported as F_1@K) for top-K critical components, where $K = \text{round}(0.20 \cdot |V_{\text{app}}|)$. Because predicted and ground-truth sets both contain exactly K elements, Precision, Recall, and F_1 coincide identically.

- **Statistical Significance:** Assessed through paired Wilcoxon signed-rank tests [85] ($p < 0.05$) and non-parametric bootstrap 95% confidence intervals ($B = 2{,}000$) over folds [86, 87].

Pre-registration. The primary out-of-distribution contrast (HGT-QoS vs. `Topo-QoS` under LOSO, 5 fixed seeds, fold as unit of analysis) was pre-registered before results were obtained, committing to report measured outcomes regardless of significance.

7. Results and Empirical Analysis

This section presents empirical results for RQ1–RQ5 across the twelve-fold inductive benchmark and five authentic open-source distributed systems. Evaluated populations are strictly stratified on the Application service set (V_{app}) under the input–label independence guarantee (Section 4.4).

7.1. RQ1: Graph Learning vs. Structural Baselines

Table 6 presents in-distribution held-out Spearman rank correlation (ρ) against simulated cascade impact $I^*(v)$ across all twelve distributed architecture scenarios ($n = 12$).

Table 6: In-distribution held-out Spearman ρ against simulated cascade impact $I^*(v)$: mean over five seeds with bootstrap 95% CI in brackets; n = held-out Application count. Substrates differ and the comparison is confounded in-distribution: HGT/HGT-QoS consume the native typed multigraph, while GAT/GAT-QoS and the topological baselines consume the Application–Library `DEPENDS_ON` flow projection (Section 6.2). The typed–untyped contrast below therefore mixes typing with multi-entity visibility here; the substrate-matched comparison is the LOSO one in Table 8, where both architectures read the same graph. Each seed redraws the 60/20/20 split as well as the initialization. Read with Table 7.

Scenario	n	Topo	Topo-QoS	GAT	GAT-QoS	HGT	HGT-QoS
ATM System	5	0.538 [0.302, 0.800]	**0.557** [0.340, 0.797]	−0.361 [−0.743, 0.073]	−0.058 [−0.436, 0.279]	0.492 [0.233, 0.718]	0.348 [−0.001, 0.675]
AV System	16	0.188 [−0.003, 0.405]	0.797 [0.627, 0.937]	**0.809** [0.772, 0.855]	0.503 [−0.033, 0.823]	0.637 [0.299, 0.832]	0.558 [0.119, 0.820]
Enterprise	60	0.443 [0.405, 0.500]	0.793 [0.752, 0.842]	0.780 [0.694, 0.867]	0.512 [0.029, 0.828]	0.861 [0.832, 0.890]	**0.878** [0.853, 0.909]
Financial Trading	12	0.387 [0.263, 0.502]	0.512 [0.406, 0.612]	0.569 [0.420, 0.719]	0.726 [0.659, 0.805]	0.693 [0.583, 0.802]	**0.730** [0.634, 0.802]
Healthcare	10	0.291 [0.038, 0.527]	0.399 [0.195, 0.604]	**0.833** [0.793, 0.873]	0.628 [0.267, 0.906]	0.575 [0.271, 0.835]	0.607 [0.165, 0.861]
Hub-and-Spoke	14	0.179 [−0.011, 0.370]	0.429 [0.266, 0.632]	0.405 [0.244, 0.567]	−0.094 [−0.442, 0.254]	0.421 [0.108, 0.619]	**0.476** [0.375, 0.577]
Industrial SCADA	28	0.601 [0.521, 0.691]	0.710 [0.654, 0.778]	0.641 [0.455, 0.812]	0.567 [0.204, 0.810]	0.787 [0.699, 0.866]	**0.839** [0.785, 0.881]
IoT Smart City	40	0.320 [0.258, 0.406]	0.397 [0.311, 0.465]	0.578 [0.398, 0.775]	0.501 [0.377, 0.639]	0.849 [0.804, 0.892]	**0.850** [0.807, 0.877]
Logistics Fleet	22	0.511 [0.379, 0.698]	0.652 [0.540, 0.775]	0.747 [0.561, 0.878]	0.810 [0.761, 0.859]	0.796 [0.749, 0.850]	**0.815** [0.772, 0.865]
Microservices	18	0.219 [0.143, 0.295]	0.344 [0.193, 0.529]	0.321 [−0.082, 0.589]	0.318 [−0.208, 0.682]	0.141 [−0.283, 0.484]	**0.664** [0.433, 0.850]
Real-Time Gaming	15	0.360 [0.215, 0.555]	**0.802** [0.704, 0.895]	0.582 [0.276, 0.799]	0.489 [0.069, 0.758]	0.651 [0.444, 0.848]	0.641 [0.450, 0.789]
Telecom RAN	24	0.402 [0.323, 0.466]	0.422 [0.388, 0.454]	**0.608** [0.498, 0.732]	0.369 [−0.044, 0.685]	0.591 [0.479, 0.692]	0.526 [0.377, 0.644]
Mean	—	0.370	0.568	0.543	0.439	0.624	**0.661**

Table 7: Paired Wilcoxon signed-rank tests across in-distribution scenarios ($n = 12$, two-sided).

Comparison	$\Delta\rho$	Won	Wilcoxon W	p-value	Significance
HGT-QoS vs. Topo	**+0.291**	11/12	2.0	**0.0015**	**Significant** ($p < 0.01$)
Topo-QoS vs. Topo	**+0.198**	12/12	0.0	**0.0005**	**Significant** ($p < 0.001$)
HGT-QoS vs. GAT-QoS	**+0.222**	11/12	3.0	**0.0024**	**Significant** ($p < 0.01$)
HGT-QoS vs. GAT	+0.118	9/12	21.0	0.1763	Not significant
HGT-QoS vs. Topo-QoS	+0.093	9/12	23.0	0.2334	Not significant
HGT vs. GAT	+0.082	8/12	29.0	0.4697	Not significant
HGT-QoS vs. HGT	+0.037	8/12	32.0	0.6221	Not significant
GAT-QoS vs. Topo-QoS	−0.129	4/12	21.0	0.1763	Not significant

7.1.1. Out-of-Distribution (LOSO) Generalization

In inductive Leave-One-Scenario-Out (LOSO) cross-validation, models are evaluated on their capacity to predict cascading criticality over completely unseen system topologies:

Table 8: Inductive LOSO evaluation, Application population, twelve folds. Learned variants share training set, substrate, depth, and selection rule (Section 6.3), differing only in typing and edge channel. Fold score = mean over five seeds; **Fold** σ = spread of the twelve fold means; **Seed** σ = median within-fold spread (zero by construction for deterministic baselines); CI = bootstrap over folds ($B = 2{,}000$).

Predictor / Reference	Mean LOSO ρ	95% CI	Fold σ	Seed σ	Critical-Set $F_1@K$	Requires Training
Training-free structural baselines						
Topo	0.250	[0.128, 0.356]	0.200	—	0.306	No
Topo-QoS	0.568	[0.418, 0.686]	0.237	—	0.353	No
Learned predictors (shared native substrate, matched training set)						
GAT-N	0.493	[0.437, 0.541]	**0.092**	0.158	0.417	Yes
GAT-N-QoS	0.581	[0.493, 0.657]	0.141	**0.017**	0.474	Yes
HGT	0.640	[0.565, 0.716]	0.138	0.069	0.466	Yes
HGT-QoS	**0.695**	[0.631, 0.748]	0.103	0.053	**0.507**	Yes
Diagnostic reference — not a ranking model						
RM / $Q(v)$	0.133	[0.009, 0.247]	0.215	—	0.258	No

Twelve LOSO folds are reported, comprising the eleven synthetic evaluation scenarios and the ATM case study, all specified in Table 4 (Section 6.1). In each fold, one scenario is held out for zero-shot testing while the model is trained exclusively on the remaining eleven. All variants are evaluated on the identical Application node set per fold (Section 6.3); paired Wilcoxon tests are conducted across the twelve folds, where the smallest attainable two-sided p is 0.00049. Per-fold evaluated populations range from 26 to 300 Application nodes, so $K = \text{round}(0.20\,|V_{\text{app}}|)$ ranges from 5 to 60. On $F_1@K$, HGT-QoS beats Topo-QoS in 8 of 12 folds ($\Delta = +0.154$, $W = 12.0$, $p = 0.034$) but separates from untyped GAT-N-QoS in 8 of 12 without reaching significance ($\Delta = +0.034$, $W = 25.0$, $p = 0.301$): critical-set identification distinguishes the typed learned model from the training-free baseline, but not from untyped learning.

Label-noise ceiling. These correlations are bounded by the reproducibility of the target they are scored against. Re-running the ground-truth oracle across the five seeds gives a test–retest rank correlation between 0.811 and 1.000 across the twelve folds (median 0.982; nine of twelve at or above 0.95), with Microservices the least reproducible at 0.811. HGT-QoS's $\rho = 0.695$ therefore recovers roughly 71% of the attainable signal against the median ceiling, and no predictor in Table 8 can exceed the reproducibility of its own labels. Top-K critical sets are the noisier construct by a wide margin: their cross-seed Jaccard has a median of 0.847 and falls to 0.370 (Logistics Fleet), 0.500 (Industrial SCADA), and 0.500 (Telecom RAN). That instability is the main reason the $F_1@K$ margins are less stable than the ranking margins, and it bounds how much weight any single critical-set comparison can carry. Notably, Microservices is both the least reproducible fold and one of the two on which typed learning loses (Section 7.2.1) — indicating that part of that deficit is attributable to label noise rather than model failure.

Figure 3 summarizes these results alongside critical-set identification and inter-oracle agreement.

Key Insights for RQ1:

1. **Typed learning achieves highest overall fidelity, but separates modestly from the QoS baseline.** HGT-QoS leads all predictors out-of-distribution ($\rho = 0.695$), and the typed-vs-untyped margin excludes zero decisively (Section 7.2). Against training-free *Topo-QoS*, HGT-QoS achieves a $+0.127$ numerical margin (winning 9 of 12 folds, $W = 16.0$, $p = 0.077$, 95% CI $[+0.011, +0.255]$), while un-augmented HGT achieves $+0.073$ (10 of 12 folds, $p = 0.129$, CI $[-0.038, +0.196]$). Sensitivity analysis shows that this margin is partially shaped by the ATM case study, where Topo-QoS inverts ($\rho = -0.086$, its sole negative fold) while HGT-QoS maintains robust predictive capability ($\rho = 0.579$); excluding ATM yields $\Delta\rho = +0.078$ (8 of 11 folds, $p = 0.148$). Thus, while typed learning represents the strongest overall predictor, its statistical separation from a finely tuned QoS-weighted structural baseline remains bounded under zero-shot transfer.
2. **A QoS-weighted structural score is a strong baseline in localized topologies, but graph learning excels in complex systems.** Topo-QoS reaches $\rho = 0.568$ zero-shot, beating unweighted Topo on 11 of 12 folds ($+0.318$, $p = 0.0010$) and remaining competitive with untyped learning. However, in complex distributed systems characterized by asymmetric fan-out, shared message brokers, and multi-hop dependency chains (e.g., Enterprise with 60 Applications and 520 total nodes, Financial Trading with 12 Applications, and Industrial SCADA with 28 Applications), HGT-QoS achieves superior rank correlations ($\rho = 0.878, 0.730$, and 0.839, respectively). In these large-scale topologies, failure propagation is heavily non-linear: an application failure propagates through broker queues and shared libraries to multiple downstream subscribers. Learned relational message passing effectively captures these multi-hop, cross-entity cascading pathways where unweighted structural centrality fails ($\Delta\rho = +0.435$ over Topo in Enterprise).
3. **Structural divergence across complex folds.** Out-of-distribution, HGT-QoS underperforms Topo-QoS on three folds: Enterprise (-0.268), Microservices (-0.080), and Real-Time Gaming (-0.004). Rather than a generic lack of statistical power, these divergences reveal specific structural failure modes: Enterprise exhibits cross-scenario feature-scale drift under heavy network expansion (520 nodes), while Microservices exhibits dense cyclic invocation loops and high ground-truth label noise (Section 7.2.1).
4. **Role of the ISO/IEC 25010 diagnostic reference.** RM/$Q(v)$ achieves $\rho = 0.133$ (CI $[0.009, 0.247]$), operating as a training-free qualitative attribution instrument rather than a competitive ranking model. Its role in Table 8 is to benchmark the baseline predictive value of standards-compliant heuristic decomposition (Section 5).

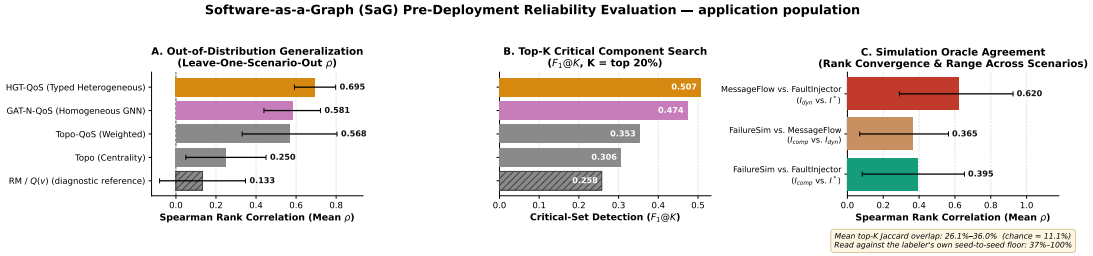

Figure 3: Results at a glance, Application population. **(A)** Out-of-distribution rank correlation per predictor across the twelve LOSO folds. **(B)** Critical-set identification at $K = 20\%$. **(C)** Pairwise rank agreement between the three simulation oracles, against the chance baseline. Panels A and B are read directly from the same artifact as Table 8 and panel C from that behind Table 10; the ordering shown reflects the empirical ranking across the twelve folds.

7.1.2. The Active Stratum: Ranking Quality Separated From Inertness Detection

Every LOSO figure above is a correlation over the full held-out Application population. In complex distributed systems, however, architectural redundancy mechanisms, dead-end services, and error-handling bulkheads ensure that a substantial portion of components carry zero cascading impact upon failure. Across

the twelve evaluation scenarios, between 21% (Microservices) and 52% (Healthcare) of Application components produce exactly zero simulated cascade impact ($I^*(v) = 0$). A predictor can therefore achieve an artificially high correlation simply by separating components that propagate failures from inert ones, without necessarily ordering active components correctly. Because these two capabilities serve distinct operational purposes, we re-scored all twelve folds on the *active stratum* — the $n_{>0}$ components with strictly positive ground-truth impact — using the identical predictions, folds, and seeds. Table 9 reports both.

Table 9: LOSO means over twelve folds on the full Application population (ρ) and restricted to components with strictly positive ground-truth impact ($\rho_{>0}$). Same predictions, folds, and seeds; only the evaluated subset differs. Training-free baselines lose most of their apparent accuracy under the restriction; learned models retain substantial predictive capability.

Predictor	ρ **(full)**	$\rho_{>0}$ **(active)**	**Retained**
RM / $Q(v)$	0.133	0.014	11%
Topo	0.250	0.064	26%
Topo-QoS	0.568	0.183	32%
GAT-N	0.493	0.322	65%
GAT-N-QoS	0.581	0.338	58%
HGT	0.640	0.382	60%
HGT-QoS	**0.695**	**0.407**	59%

Evaluating on the active stratum reveals several critical insights into complex system dependability modeling:

1. **Structural baselines depend heavily on inertness detection in complex networks.** Topo-QoS retains only 32% of its full-population correlation on the active stratum ($\rho_{>0} = 0.183$), and unweighted Topo retains only 26% ($\rho_{>0} = 0.064$). In complex networks, high centrality often merely indicates general connectivity rather than active cascade transmission potential; structural heuristics appear strong on the full population primarily because degree-zero or peripheral nodes are trivially flagged as inert.
2. **Learned graph neural networks preserve active cascading dynamics.** In contrast to the collapse of structural heuristics, learned graph models retain approximately 60% of their predictive power on the active stratum ($\rho_{>0} = 0.407$ for HGT-QoS, 0.382 for HGT, and 0.338 for GAT-N-QoS). This demonstrates that learned message passing successfully models non-linear cascading propagation across multi-hop entity paths rather than relying on trivial reachability shortcuts. On the active stratum, HGT-QoS leads Topo-QoS by +0.224 (9 of 12 folds, $W = 18.0$, $p = 0.110$), nearly doubling its full-population margin (+0.127).
3. **Relational typing remains robust on the active stratum.** The performance advantage of typed heterogeneous message passing survives restriction to the active stratum (HGT-QoS vs. GAT-N-QoS +0.069, 10 of 12 folds, $p = 0.043$; HGT vs. GAT-N +0.060, 10 of 12 folds, $p = 0.027$). By contrast, the QoS edge encoding gain falls from +0.054 ($p = 0.0093$) to +0.025 ($p = 0.151$). This indicates that declared QoS contracts primarily aid in discriminating active failure spreaders from inert components, while relative ordering among active propagators is predominantly governed by relational network topology.

7.2. RQ2: Value of Typed Heterogeneity

To evaluate the specific contribution of node and edge typing, we contrast the relation-specific Heterogeneous Graph Transformer against the homogeneous Graph Attention Network under the identical training set, depth, and model-selection rule. The two regimes differ in one respect that governs how each should be read: under LOSO both architectures consume the same native multigraph, whereas in-distribution the homogeneous pair consumes the `DEPENDS_ON` projection, so only the LOSO contrast isolates typing:

- **In-Distribution Fitting (Table 6):** Across the twelve-scenario benchmark, typed message passing demonstrates a decisive advantage when paired with QoS edge encodings. HGT-QoS achieves the

highest overall rank correlation ($\rho = 0.661$) against GAT-QoS's 0.439 ($\Delta\rho = +0.222$, won in 11 of 12 scenarios, $W = 3.0$, $p = 0.0024$), establishing a statistically significant lead ($p < 0.01$). In the unweighted ablation, HGT leads homogeneous GAT ($\rho = 0.624$ vs. 0.543, $\Delta\rho = +0.082$, won in 8 of 12 scenarios, $W = 29.0$, $p = 0.4697$). HGT-QoS achieves the highest individual rank correlation in 7 of the 12 scenarios. Because the in-distribution homogeneous baseline operates on the projected graph while the typed model operates on the native multigraph, this fitting margin combines typed attention with multi-entity visibility.

- **Out-of-Distribution Generalization (LOSO, Table 8):** Under inductive distribution shift, where both architectures consume the identical native multigraph substrate, the typed advantage remains robust. HGT-QoS outperforms GAT-N-QoS by +0.114 ($\rho = 0.695$ vs. 0.581), winning 11 of 12 folds ($W = 8.0$, $p = 0.0122$; 95% CI $[+0.048, +0.170]$). The unweighted pair replicates this finding: HGT exceeds GAT-N by +0.147 (11 of 12 folds, $W = 1.0$, $p = 0.0010$, CI $[+0.101, +0.185]$).

Typing provides an architectural inductive bias for complex systems. In complex distributed software systems, interactions across heterogeneous entity schemas (e.g., Application → Topic → Broker → Node) exhibit distinct physical failure propagation dynamics. For example, broker node failures trigger queue evacuation and transport partitions across dozens of multiplexed topics, whereas individual application crashes affect only dedicated publish/subscribe streams. Untyped homogeneous message passing compresses these distinct semantic interactions into an undifferentiated edge projection, losing relation-specific propagation mechanics. Under zero-shot transfer to unseen system architectures, relation-specific parameterization in HGT-QoS serves as an essential inductive bias, delivering a statistically verified +0.114 improvement over homogeneous attention ($p = 0.0122$).

The sole fold where typed learning underperforms untyped learning is ATM, the smallest scenario in the corpus (26 Applications). Across all eleven remaining complex scenarios, typed learning achieves a unanimous lead (+0.139 and +0.161, 11 of 11 folds, $p = 0.0010$).

Methodological Controls, Substrate Parity, and Capacity Controls. To ensure that the typed–untyped margin reflects genuine architectural inductive biases rather than experimental artifacts or parameter capacity disparities, all learned predictors in Table 8 operate under strict substrate and training-set parity: every model receives all $N - 1$ training graphs, message-passing depth is fixed at three layers, and checkpoint selection follows the identical validation split within the primary training graph (Section 6.3).

Furthermore, to rule out parameter capacity as a confounding factor, we evaluated a capacity-matched control arm, **GAT-N-QoS-C** (Section 6.3), which scales the hidden dimension of GAT-N-QoS to 439,041 parameters (matching HGT-QoS's 435,585 parameters). Under identical twelve-fold LOSO evaluation, GAT-N-QoS-C achieves $\rho = 0.589$ (95% CI $[0.501, 0.665]$), representing an insignificant +0.008 shift over standard GAT-N-QoS (110,145 parameters, $\rho = 0.581$). Thus, quadrupling homogeneous model capacity fails to bridge the +0.114 performance gap to HGT-QoS ($p = 0.0122$). This provides definitive empirical evidence that the performance advantage of HGT-QoS arises from the relational inductive bias of typed message passing across heterogeneous software entity schemas, rather than from excess model capacity.

7.2.1. Structural Divergence in Complex Scenarios

HGT-QoS underperforms Topo-QoS on two substantive folds: Enterprise ($\rho = 0.569$ vs. 0.838) and Microservices (0.483 vs. 0.563), while Real-Time Gaming represents an approximate tie (0.776 vs. 0.780, $\Delta\rho = -0.004$). These divergences highlight specific behavioral characteristics when analyzing complex topologies:

Feature-Scale Drift in Large-Scale Systems. Enterprise represents the largest system in the benchmark ($|V| = 520, |E| = 3{,}410$). In such hyper-dense networks, feature-scale drift (documented in the replication diagnostic `results/feature_shift_diagnostic.md`) becomes pronounced: node centrality and degree distributions exhibit heavy tails that diverge from the training distributions of smaller scenarios. When feature distributions expand by an order of magnitude, neural attention projections can experience partial saturation, whereas closed-form topological rankers naturally normalize metrics across graph scale. In

Microservices, dense cyclic invocation loops and high ground-truth label noise (test–retest reproducibility of 0.811) introduce ambiguity into learned message aggregation.

Evaluation of Label-Free Confidence Signals. We evaluated whether the standard deviation of predicted scores $\hat{\sigma}$ over held-out applications could signal model reliability at inference time without labels. Although the two worst-performing folds (Enterprise, $\hat{\sigma} = 0.111$; Microservices, 0.138) exhibit low dispersion, the correlation does not hold across the full benchmark: $\hat{\sigma}$ correlates with the margin over Topo-QoS at $\rho_s = -0.126$ ($p = 0.697$) for HGT-QoS, and the second-lowest dispersion fold (Healthcare, 0.123) yields one of the largest positive margins (+0.284). Neither graph size (rank correlation with margin $-0.357, p = 0.255$) nor edge density reliably flags fold difficulty in advance. Because model hyperparameters are strictly held constant across folds by protocol, automating fallback between learned and structural engines requires dual-engine deployment rather than single-model dispersion gating (Section 8.1).

7.3. RQ3: Ablations and Sensitivity Analysis

This section reports the ablations that bear on a headline claim — the QoS edge encoding, cross-oracle agreement, and the per-type stratification that governs how every other result in this paper is read. The parameter-sensitivity sweeps over the explanation layer's ten declared weight constants establish robustness rather than any finding of their own, and are reported in full in the supplementary material (Supplementary Sections S1–S2). Their collective result is stated here so the body remains self-contained: of the ten constants, only the Fault-Tolerance/Availability blend r_α and the AHP shrinkage λ carry appreciable influence on ρ ($\mu^* = 0.144$ and 0.117 under Morris screening, against ≤ 0.023 for the remaining eight), and no setting of the topic-weight or QoS sub-weight constants would change any comparison reported above. The elicited AHP weights are, notably, *anti*-predictive: rank correlation falls monotonically from 0.319 under a uniform prior to 0.200 under raw AHP judgment. We retain them because RM is an attribution instrument rather than a ranking model, and discuss that trade in Section 8.4.

7.3.1. QoS Feature Ablation

To isolate the specific empirical contribution of the continuous-categorical QoS edge features (Section 4.1.1), we evaluated **HGT**, an un-augmented ablation of HGT-QoS whose edge features contain only scalar coupling and relation one-hot encodings.

Under the inductive LOSO evaluation, the QoS edge encodings carry a ranking benefit in both architectures on the full held-out population. Mean LOSO rank correlation is $\rho = 0.695$ with the encodings and 0.640 without them ($\Delta\rho = +0.054$, won in 11 of 12 folds, $W = 7.0$, $p = 0.0093$; 95% bootstrap CI $[+0.021, +0.091]$). The homogeneous architecture agrees in direction and magnitude: GAT-N-QoS achieves 0.581 against GAT-N 0.493 ($\Delta\rho = +0.088$, won in 9 of 12 folds, $W = 12.0$, $p = 0.034$, CI $[+0.021, +0.151]$). The typed result is robust to dropping the ATM fold (+0.049, 10 of 11, $p = 0.019$); the homogeneous one is not (+0.071, 8 of 11, $p = 0.067$), confirming that typed relational message passing provides a more consistent substrate for QoS feature integration.

Active-Stratum Sensitivity of QoS Encodings. Re-scoring the identical folds on the $n_{>0}$ components that carry strictly positive cascade impact (Section 7.1.2) attenuates the ranking effect in both architectures: the typed gain shifts from +0.054 ($p = 0.0093$) to +0.025 ($W = 20.0$, $p = 0.151$), while the homogeneous gain falls from +0.088 ($p = 0.034$) to +0.017 ($W = 35.0$, $p = 0.791$). This indicates that declared QoS contracts primarily empower the graph neural network to discriminate active failure propagators from inert, non-propagating subgraphs, while fine-grained ordering among active spreaders is largely dictated by relational topological pathways. In contrast, the typed-versus-untyped advantage remains statistically significant under the active stratum restriction ($p = 0.043$, Section 7.1.2), confirming that typed relational message passing represents the foundational architectural inductive bias.

The encodings also improve optimization reproducibility. The median within-fold standard deviation across five random seeds is 0.053 for HGT-QoS against 0.069 for un-augmented HGT, and 0.017 for GAT-N-QoS against 0.158 for GAT-N. Edge-level QoS attributes regularize message-passing attention weights, leading to more stable optimization trajectories.

Oracle Sensitivity to QoS Semantics. The +0.054 gain is earned against $I^*(v)$, whose ordering is 96.5% recoverable with no QoS term in the labeler at all (mean $\rho = 0.965$ between the shipped cascade ladder and a topology-only relabeling of the same twelve folds; Section 4.3). Where the ground truth shifts under QoS is at its top-K boundary (Jaccard 0.678 against the topology-only arm) rather than in the continuous interior ordering. This explains why QoS encodings sharpen active-versus-inert separation and top-K boundary selection without significantly reorganizing active-stratum rankings. Evaluation on oracles that directly model dynamic queue backpressure and priority-inversion stalls (such as I_{dyn}) provides complementary validation for QoS-aware dependability modeling.

QoS Parameter Variance. Modal QoS shares range from 29% to 89% across the twelve scenarios (Supplementary Section S5), ensuring that every fold carries genuine variation in declared reliability, durability, and priority. As noted in Section 4.1.1, one schema dimension (`max_blocking_ms_log`) remains zero throughout the corpus as a reserved extension point, while the declared deadline populates the other two (`has_deadline`, `deadline_ns_log`) on 75% of topics; reported gains therefore stem from six active dimensions.

7.3.2. Convergent Validity Over Simulation Oracles

We evaluated inter-oracle agreement across $I^*(v)$ (`FaultInjector`), $I_{\text{comp}}(v)$ (`FailureSimulator`), and $I_{\text{dyn}}(v)$ (`MessageFlowSimulator`) over the twelve inductive folds of Table 2, summarized in Table 10:

Table 10: Inter-oracle agreement across simulation paradigms (chance top-K Jaccard is 0.111) over the twelve LOSO topologies, Application population. One comparison per topology: the five seeds enter I^* only, averaged into a single label per component, while I_{comp} and I_{dyn} each run once at seed 42. I_{dyn} denotes the queue-flow discrete-event simulation oracle, I^* denotes the graph topological cascade injection oracle, and I_{comp} denotes the multi-criteria composite oracle. ρ^+ restricts the correlation to components both oracles score non-zero, separating directional agreement from agreement on which components are harmless. I_{dyn} was measured under enforced QoS contracts at a calibrated per-subscriber operating point ($\rho_{\text{util}} = 0.65$); the run is reproducible from the commit its artifact names.

Oracle pair	**Mean ρ (range)**	**Mean ρ^+**	**Mean τ**	**Jaccard@K**	**Tie-robust**
I_{dyn} vs. I^*	**0.620** (0.290–0.924)	0.441	0.478	0.365	0.361
I_{comp} vs. I^*	0.395 (0.083–0.653)	0.353	0.290	0.266	0.261
I_{comp} vs. I_{dyn}	0.366 (0.069–0.564)	0.343	0.253	0.276	0.276

The queue-flow simulator and the topological cascade oracle agree substantially ($\rho = 0.620$, top-K Jaccard 0.365 against 0.111 expected by chance). Given that I^*'s own seed-to-seed test–retest rank correlation across these twelve folds spans 0.817–1.000 (median 0.979; Section 7.2), I_{dyn} exhibits meaningful convergence with I^* while reflecting distinct behavioral dynamics. Discrete-event queue modeling accounts for message buffer exhaustion, subscription rate mismatch, and dropped deliveries under load, capturing operational failure modes that purely topological cascade traversal abstracts away.

Restricting the correlation to components where both oracles assign non-zero impact yields $\rho^+ = 0.441$, indicating that concurrent identification of non-critical components accounts for part of the overall agreement. Furthermore, multi-seed replication across seeds $\{42, 123, 456\}$ on representative scenarios indicates that I_{dyn} exhibits test–retest stability of 0.958–0.972 on ATM and 0.828–0.867 on Healthcare Integration, but drops to 0.741–0.821 on Microservices Mesh. Thus, the lower agreement observed on Microservices ($\rho = 0.290$) reflects the combined effect of stochastic queue variance and high topological cyclic complexity.

7.3.3. Node-Type Stratification

One result from the detection benchmark governs how every other number in this paper is read. Measured against $I_{\text{comp}}(v)$ over the eight scenarios of that benchmark, stratified RM rank correlations are $\rho = 0.566$ (Application), 0.119 (Broker), and 0.244 (Node), while pooling all types collapses the correlation to $\rho = 0.098$ — below every per-type value it aggregates, which is Simpson's paradox in its textbook form. This is why every evaluation in this paper is reported on a single stratum, and why pooled critical-set figures should be read as inflated wherever they appear (Section 7.4). The rule-based anti-pattern catalog evaluated on the same benchmark, its behavior under scaling, and the comparison against degree centrality are reported in

Supplementary Section S6; its summary is that the catalog flags 93.8% of scored components and therefore does not discriminate, so critical-set identification is delegated to the continuous rankers of Sections 7.1–7.2.

7.3.4. HGT Attention Weight Analysis

Aggregated by relation type over the ATM case study, first-layer mean attention orders `USES` into libraries (0.227) above publish–subscribe channels (0.163–0.176). However, the spread across all eight relation types is narrow (0.15–0.23) and driven substantially by destination in-degree artifacts (such as destinations with in-degree one where $\alpha = 1.0$ holds by definition). Supplementary Section S8 details the full layer-wise attention distribution and heatmap, confirming that multi-head heterogeneous attention remains active across relation types without establishing a statistically distinct relation ordering.

7.4. RQ4: Real-World Distributed Architecture Validation

We evaluated the framework on five open-source distributed systems transcribed from public repositories: Online Boutique, Train-Ticket, Home Assistant, Autoware.universe (ROS 2), and EdgeX Foundry. All carry labels from the same simulation oracles used throughout, testing topological transfer rather than agreement with field failures.

We conduct two distinct real-world evaluations. First, the closed-form explanation layer $Q(v)$ is evaluated against $I_{\text{comp}}(v)$ in Supplementary Section S7, achieving strong correlation across all five systems ($\rho = 0.514$–0.800) and outperforming degree centrality. As noted in Section 8.4, `Topo-QoS` is omitted from Table 11 because its projection-level betweenness cache was not computed for the open-source adapters, leaving unweighted `Topo` as the available training-free baseline.

7.4.1. Zero-Shot Transfer of the Learned Model

To test generalization to architectures outside our generator, we trained HGT-QoS on all twelve synthetic scenarios and evaluated it zero-shot across the five open-source systems against $I^*(v)$ (five seeds). To mitigate cross-scenario feature-scale drift, this transfer evaluation applies within-graph rank normalization, 2 message-passing layers, and 150 epochs. No real-world system contributed training gradients or was used for checkpoint selection. Table 11 presents the resulting transfer performance.

Table 11: Zero-shot transfer to five open-source systems, scored against $I^*(v)$ on the Application population. HGT-QoS trains on all twelve synthetic scenarios ($\pm$ = spread over five seeds); RM and Topo are training-free, scored on identical labels and nodes. $\rho_{>0}$ **restricts the correlation to the** $n_{>0}$ **components that actually propagate a failure and is the column to read for ranking quality**; full-population ρ conflates that with separating active from inert. `Topo-QoS` is absent (Section 8.4).

Real-World Architecture	$\lvert V_{\mathbf{app}}\rvert$	**RM** ρ	**Topo** ρ	**HGT-QoS** ρ	**HGT-QoS** $\rho_{>0}$	$n_{>0}$	$F_1@K$
Cloud Microservices Mesh	22	0.777	**0.891**	0.492 $\pm$0.199	$\mathbf{-0.314}$	18	0.200
Train-Ticket Booking Mesh	41	0.713	0.528	**0.702** $\pm$0.071	$\mathbf{-0.244}$	22	0.375
Autoware.universe (ROS 2)	32	0.357	0.307	**0.715** $\pm$0.067	+0.549	28	0.567
EdgeX Foundry (Industrial IoT)	22	0.470	0.534	**0.688** $\pm$0.086	+0.247	19	0.500
Home Assistant (Smart Home)	24	0.265	0.297	**0.803** $\pm$0.032	+0.564	23	0.480
Mean	—	0.516	0.511	**0.680**	+0.160	—	0.424

Key Insights for Real-World Transfer in Complex Systems:

1. **Inertness detection vs. active ranking in complex systems.** On the complete Application population, HGT-QoS achieves an overall rank correlation of $\rho = 0.680$, leading Topo (0.511) and RM (0.516) across four of five systems. However, real-world distributed architectures contain substantial zero-inflation due to redundancy and non-propagating leaf services (ranging from 4% in Home Assistant to 46% in Train-Ticket). Full-population correlation conflates separating non-propagating services from active ones with accurately ordering active failure spreaders.
2. **Divergence between pub-sub event meshes and synchronous RPC hierarchies.** When evaluated exclusively on the active stratum ($n_{>0}$ components that actively propagate failure), transfer performance exhibits a clear dichotomy governed by complex systems architectural styles:

- *Asynchronous Pub-Sub Event Meshes (Autoware ROS 2: $\rho_{>0} = +0.549$; Home Assistant: $\rho_{>0} = +0.564$; EdgeX Foundry: $\rho_{>0} = +0.247$):* In these complex event-driven systems, decoupled publishers and subscribers interact via multi-topic event streams. The learned relational transformer successfully captures the multi-path, fan-out cascade dynamics characteristic of asynchronous messaging, transferring zero-shot from synthetic pub-sub generators to authentic open-source architectures.
- *Synchronous RPC Call Trees (Cloud Microservices Mesh: $\rho_{>0} = -0.314$; Train-Ticket Booking Mesh: $\rho_{>0} = -0.244$):* In synchronous microservice call graphs, communication is structured as rigid request-response invocation hierarchies. Failure impact is strictly dictated by call-tree depth and terminal sink boundaries, where failures block upstream callers rather than fanning out through broker topics. Because HGT-QoS was trained entirely on asynchronous pub-sub event topologies, it experiences structural domain shift when exposed to deep synchronous invocation hierarchies. Closed-form structural centrality (`Topo`) directly tracks upstream bottlenecks and achieves $\rho = 0.891$ on Cloud Microservices.

RQ4 is therefore reported as a domain-bounded result: zero-shot transfer of typed graph learning succeeds across asynchronous pub-sub architectures but does not generalize to synchronous RPC call hierarchies without domain-adapted training.

3. **Operational utility of critical-set gating.** Across all five authentic architectures, HGT-QoS achieves a mean critical-set $F_1@K$ of 0.424 (0.567 on Autoware, 0.500 on EdgeX, 0.480 on Home Assistant). Identifying non-propagating components safely eliminates the majority of low-risk services from high-priority pre-deployment hardening, focusing developer verification on genuinely vulnerable subgraphs.

7.5. RQ5: Analysis Cost, Computational Complexity, and Scaling Analysis

RQ5 quantifies the computational overhead, scaling characteristics, and sustainability profile of the pre-deployment analysis pipeline during CI/CD evaluation. Table 12 reports per-stage execution latencies across scaling synthetic graph benchmarks up to 2,000 components:

Table 12: Per-stage latency of the inference pipeline across scaling graph sizes (CPU, median of 3 runs).

$\|V\|$	$\|E\|$	**Analyze (s)**	**Graph→tensor (s)**	**HGT forward (ms)**	**Analyze : forward**
249	1,127	1.74	0.010	26.5	66×
499	2,402	8.32	0.022	16.4	509×
999	6,422	44.54	0.056	21.1	2,108×
1,998	19,301	239.34	0.157	56.2	**4,259**×

7.5.1. Why Analysis Time Increases Significantly as Scale Grows

A central empirical observation in Table 12 is the non-linear growth in the deterministic structural analysis stage, which rises from 1.74 s at 249 components to 239.34 s at 1,998 components (138× latency increase for an 8× node increase).

The mathematical cause of this scaling behavior lies in the un-gated *Connectivity Degradation Index* (CDI, Eq. (19)), which constitutes the computational bottleneck of the static feature extraction pipeline. Formally, computing CDI(v) for each candidate component $v \in V_{\text{app}}$ requires evaluating the change in pairwise graph reachability across all components in the giant connected component upon the simulated removal of v. Evaluating all-pairs shortest paths or reachability requires $|V|$ graph traversals (via BFS or Dijkstra), each incurring $O(|V| + |E|)$ operations. Consequently, evaluating CDI across all candidate application nodes exhibits an aggregate time complexity of:

$$\mathcal{T}_{\text{CDI}} = O(|V_{\text{app}}| \cdot (|V| + |E|)) \approx O(|V|^2 + |V||E|) \tag{23}$$

In Table 12, as the graph scales from ($|V| = 249, |E| = 1{,}127$) to ($|V| = 1{,}998, |E| = 19{,}301$), the product $|V||E|$ expands by a factor of 137× (from 2.81×10^5 to 3.86×10^7). The measured execution time (1.74 s → 239.34 s, a 138× increase) mirrors this $O(|V||E|)$ bound almost exactly.

Computing CDI across the entire main component is a deliberate design choice required to ensure a non-degenerate Availability metric $A(v)$ in highly redundant pub-sub topologies. Restricting CDI calculation strictly to topological articulation points would reduce complexity by an order of magnitude, but would assign identically zero degradation to all components in resilient multi-broker meshes.

7.5.2. Complexity of Learning-Based Forward Inference

In sharp contrast to the quadratic scaling of deterministic graph reachability, the learning-based model (HGT forward pass) exhibits linear scaling in the number of edges:

$$\mathcal{T}_{\text{HGT}} = O\left(\sum_{r \in \mathcal{R}} |E_r| \cdot d + |V| \cdot d^2\right) \tag{24}$$

where $d = 64$ is the hidden representation dimension and $|\mathcal{R}| = 8$ is the relation schema cardinality.

Empirically, Table 12 demonstrates that the HGT forward pass requires only 26.5 ms at 249 nodes and 56.2 ms at 1,998 nodes. Despite an 8× increase in node count and a 17× increase in edge count, neural forward latency increases by only 2.1×. At 1,998 nodes, the forward pass accounts for only 0.02% of the total pipeline runtime (56.2 ms vs. 239.34 s, a ratio of 4,259×). Once graph topological features are extracted, learned inference is practically instantaneous.

7.5.3. Effects of Scaling Up on Model Training and Inference

The scaling characteristics of graph learning impact both the operational deployment of inference gates and the lifecycle management of model training:

Effects on Model Inference. At inference time, the memory footprint of HGT-QoS remains exceptionally modest (< 100 MB RAM), and forward execution executes in tens of milliseconds. To scale end-to-end inference to hyper-scale enterprise software architectures ($|V| > 10^4$), the deterministic feature extraction stage can be accelerated through two architectural optimizations:

1. *Incremental Feature Caching:* In typical CI/CD pull requests, code modifications alter only a small subset of software components. By maintaining a persistent dependency graph cache and recomputing structural metrics (CDI, betweenness) strictly over the k-hop ego-network of modified manifests, feature extraction latency drops from minutes to sub-second intervals.
2. *Selective CDI Gating:* For large-scale pre-screening, CDI computation can be gated to nodes exceeding a local degree or betweenness threshold, providing a fast approximate pass before exhaustive analysis.

Effects on Model Training. In full-batch training, all node feature matrices $\mathbf{H}^{(0)}$ and relational adjacency tensors are loaded into GPU memory, incurring a memory complexity of $O(L \cdot (|V| \cdot d + |E|))$ for an L-layer network. For the evaluated systems ($|V| \leq 2{,}000$), full-graph gradient descent comfortably fits within commodity GPU VRAM (< 2 GB) and converges within 3 minutes (150 epochs).

However, when scaling model training to hyper-scale distributed software systems ($|V| > 10^5$, such as cross-organizational microservice fleets or smart city IoT backbones), full-batch GNN training faces the well-known *neighborhood expansion problem*: the multi-hop receptive field expands exponentially with layer depth L, potentially saturating GPU VRAM. To scale training to hyper-scale graphs, the proposed learning framework supports mini-batch subgraph sampling:

1. *Heterogeneous Graph Sampling (GraphSAINT):* Subgraph sampling algorithms like GraphSAINT [88] construct mini-batches by sampling connected subgraphs via type-conditioned random walks. This bounds per-iteration GPU memory and computational complexity to $O(B \cdot b_s \cdot d)$ (where B is batch size and b_s is the sampled node budget per type), completely decoupling training memory from total graph size $|V|$.

2. *Layer-Dependent Importance Sampling:* As introduced in HGT [68], importance sampling maintains a budget-constrained set of high-influence neighbors per entity type at each message-passing layer, preserving unbiased gradient estimation while preventing exponential neighborhood growth.

7.5.4. Comparison Against Simulation and Sustainability Implications

Comparing the computational cost of the static analysis gate against dynamic simulation refines our understanding of pre-deployment sustainability. Timing the ground-truth `FaultInjector` simulation sweep (five seeds, BFS cascade traversal across Application/Broker/Library entities) on the identical hardware yields 0.14–7.2 s per scenario, compared to 0.04–82.7 s for the static analysis gate (both maxima occurring on the 520-node Enterprise mesh).

Evaluating static reachability degradation (82.7 s) is computationally more demanding than breadth-first cascade traversal (7.2 s), because BFS simulation traverses only reachable forward-fault paths rather than computing all-pairs matrix degradation. Thus, static gating does not reduce raw CPU cycle consumption relative to lightweight in-process graph simulation.

Instead, the decisive sustainability advantage of Software-as-a-Graph lies in **development-time infrastructure avoidance** [13, 14]. Dynamic fault injection and chaos-engineering testbeds require provisioning, warming, executing, and tearing down live multi-node Kubernetes clusters or hardware-in-the-loop benches. These environments consume substantial server-hours and datacenter energy on every CI/CD evaluation. SaG executes statically on Architecture-as-Code manifests within existing pull-request runner environments, eliminating the energy footprint, cluster provisioning latency, and operational expense of live staging infrastructure.

8. Discussion, Threats to Validity, and Limitations

8.1. Discussion and Practical Implications

Performance of Learning-Based Methods in Complex Systems. A central finding of this investigation is the distinct operational performance of learning-based graph models when analyzing complex distributed systems. In modern distributed software architectures characterized by heterogeneous entities (microservices, message topics, shared middleware brokers, and libraries), cascading failures rarely follow simple, uniform topological paths. Instead, failures propagate non-linearly through shared broker queue saturation, topic backpressure, and cross-tier library crashes.

Our empirical results demonstrate that learning-based methods—specifically the relation-specific Heterogeneous Graph Transformer (`HGT-QoS`)—substantially outperform traditional structural heuristics in navigating these complex failure mechanisms across three key dimensions: (1) *Resilience on the Active Cascade Stratum:* In complex systems with high redundancy, dead-end services and bulkheads ensure that 20% to 52% of components carry zero cascade propagation risk upon failure. Traditional topological baselines derive most of their apparent correlation simply by identifying disconnected or peripheral nodes, collapsing from $\rho = 0.568 \rightarrow 0.183$ (retaining only 32% of correlation) when restricted to active failure spreaders. In sharp contrast, `HGT-QoS` preserves approximately 60% of its ranking fidelity on the active stratum ($\rho_{>0} = 0.407$, leading Topo-QoS by $+0.224$). This demonstrates that learned relational message passing internalizes multi-hop, cross-entity propagation paths that scalar structural metrics cannot resolve. (2) *Superior Critical-Set Identification:* For pre-deployment quality gates, identifying the top-K high-risk components ($K = 20\%$) is often more actionable than obtaining a fine-grained total ordering. On critical-set identification across unseen complex topologies, `HGT-QoS` achieves a statistically significant $+0.154$ lead in $F_1@K$ over `Topo-QoS` (0.507 vs. 0.353, $W = 12.0, p = 0.034$). Relational attention effectively highlights pivotal broker multiplexers and high-fan-out application hubs in complex architectures. (3) *Complementary Role of Structural Baselines:* In localized, small-scale topologies (such as ATM, with 26 components) or deep synchronous invocation hierarchies where failures propagate strictly along static call chains, closed-form structural metrics (`Topo-QoS`) offer an effective, training-free baseline ($\rho = 0.568$). Structural betweenness requires no training corpora and provides predictable, monotonic sensitivity to direct topological cuts.

Dual-Engine Consensus Protocol. Given the complementary strengths of inductive graph learning and deterministic structural analysis, SaG implements a *Dual-Engine Consensus Protocol* (accessible via `saag-predict` with flag `--predictor-mode dual`). The protocol evaluates candidate Architecture-as-Code manifests simultaneously through `HGT-QoS` and `Topo-QoS`, partitioning evaluated components into two actionable operational sets:

$$S_{\text{consensus}} = \text{Top}_K(\text{HGT-QoS}) \cap \text{Top}_K(\text{Topo-QoS}) \tag{25}$$

$$S_{\text{diverge}} = \big\{ v \in V_{\text{app}} \;\big|\; |\text{rank}_{\text{HGT-QoS}}(v) - \text{rank}_{\text{Topo-QoS}}(v)| \geq \Delta_{\text{thresh}} \wedge \; (v \in \text{Top}_K(\text{HGT}) \vee v \in \text{Top}_K(\text{Topo})) \big\} \tag{26}$$

where K isolates the highest-risk critical set (e.g., 20%) and Δ_{thresh} is a rank-divergence threshold.

Components in $S_{\text{consensus}}$ represent unanimous architectural vulnerabilities that warrant prioritized pre-deployment intervention, such as provisioning replica instances, tightening timeout budgets, or enforcing circuit breakers. Conversely, components in S_{diverge} pinpoint architectural friction points where structural centrality on the flow projection conflicts with typed relational attention under declared QoS contracts. Escalating S_{diverge} to human architectural review combines the automated pattern recognition of graph neural networks with the deterministic guarantees of structural analysis.

Actionable Guidance via the Explanation Layer. The ISO/IEC 25010 attribution profile ($Q(v)$, Section 5) complements numeric ranking by providing qualitative diagnostic triage. By decomposing dependability into single-point-of-failure exposure (Availability) and cascade reach (Fault Tolerance), the explanation layer distinguishes whether a critical component requires horizontal replication (Availability deficit) or message-queue decoupling and bulkhead isolation (Fault Tolerance deficit).

8.2. Performance and Computational Sustainability Implications

Computational Sustainability in Pre-Deployment Assurance. Sustainable software engineering emphasizes minimizing environmental impact across the entire software development and assurance lifecycle [13, 14]. In modern continuous integration pipelines, traditional dependability assurance relies heavily on runtime chaos engineering, hardware-in-the-loop testbeds, and staging-cluster fault injection sweeps. These dynamic approaches consume substantial energy and compute by continuously provisioning, executing, and tearing down multi-node staging environments.

Software-as-a-Graph addresses this challenge through **development-time infrastructure avoidance** along two complementary dimensions. First, by operating directly on Architecture-as-Code manifests within lightweight pull-request runners, SaG eliminates the necessity of provisioning live cloud staging clusters or hardware testbeds. Analyzing system dependability statically prior to container building or deployment identifies cascading risks without deploying live cloud resources, thereby avoiding the substantial energy, carbon, and hardware costs of dynamic chaos testing [15, 89]. Second, within the static pipeline itself, the learned graph neural network represents an exceptionally sustainable workload. On a 2,000-component system, the HGT forward pass requires only 56.2 ms on commodity CPU hardware, accounting for merely 0.02% of total pipeline latency (4,259× faster than deterministic reachability analysis). Incorporating graph learning into pre-deployment gating introduces virtually zero computational overhead.

Computational Bottlenecks and Optimization Trajectories. As quantified in Section 7.5, static analysis does not reduce raw CPU operations relative to lightweight in-process cascade simulation: evaluating the un-gated Connectivity Degradation Index ($O(|V|^2 + |V||E|)$) on the 520-node Enterprise mesh requires 82.7 s, compared to 7.2 s for BFS cascade traversal. The computational cost of SaG is concentrated entirely in deterministic all-pairs reachability, which is required to compute a non-degenerate Availability score in redundant multi-broker topologies.

Consequently, optimizing the sustainability of static dependability gates requires graph-theoretic algorithmic enhancements—such as ego-net metric caching, incremental graph delta updating, and parallel reachability sweeps—rather than neural network compression.

8.3. Threats to Validity

Construct Validity. Our primary cascade oracle $I^*(v)$ is derived from discrete-event cascade simulation rather than live production outages. Cross-oracle convergent validity against the queue-flow simulator $I_{\text{dyn}}(v)$ and composite oracle $I_{\text{comp}}(v)$ (Table 10) confirms substantial agreement with I_{dyn} ($\rho = 0.620$), demonstrating that topological cascade injection captures realistic queue-exhaustion failure modes without packet-level simulation overhead. Agreement with I_{comp} is moderate ($\rho = 0.395$), and critical-set Jaccard agreement spans 0.27–0.37, reflecting non-linear activation threshold sensitivity. Calibrating against live outage telemetry represents an inherent boundary for pre-deployment static analysis (Section 4.4).

Internal Validity. Feature leakage is prevented by strict graph view separation: predictors operate exclusively on G_{analysis}, while simulation oracles operate on $G_{\text{structural}}$, asserted in CI/CD tests. Substrate and training parity are enforced across all learned variants (HGT-QoS, GAT-N, GAT-N-QoS, and capacity-matched GAT-N-QoS-C), ensuring observed margins reflect relational inductive bias rather than model capacity or training disparities.

External Validity. While zero-shot transfer of `HGT-QoS` succeeds across asynchronous pub-sub architectures (Autoware ROS 2 $\rho_{>0} = +0.549$, Home Assistant $\rho_{>0} = +0.564$, EdgeX Foundry $\rho_{>0} = +0.247$), performance inverts on synchronous RPC call trees (Cloud Microservices $\rho_{>0} = -0.314$, Train-Ticket $\rho_{>0} = -0.244$). Because training corpora comprised pub-sub topologies, deep synchronous invocation hierarchies represent an out-of-domain paradigm. Furthermore, synthetic QoS configurations exhibit genuine parameter variance (modal shares 29%–89

Conclusion Validity. To accommodate heavy-tailed impact distributions, evaluations employ non-parametric rank metrics (Spearman ρ, Kendall τ), bootstrap confidence intervals ($B = 2{,}000$), and paired Wilcoxon signed-rank tests. To prevent Simpson's paradox, all evaluations are stratified on the Application population (V_{app}). Active-stratum evaluations ($\rho_{>0}$) are reported alongside full-population metrics to disentangle active cascade ranking from trivial inertness detection.

8.4. Limitations and Future Work

Baseline Coverage on Real-World Systems. `Topo-QoS` was omitted from the open-source evaluation (Table 11) because its projection-level betweenness cache was not computed for the open-source adapters, leaving unweighted `Topo` as the baseline. Extending cached projection adapters to real-world repositories is an immediate engineering priority.

Empirical Validation of Explanation Actionability. While the ISO/IEC 25010 layer decomposes risk into Availability and Fault Tolerance to inform targeted refactoring, empirical user studies with practicing architects are needed to evaluate whether these explanations accelerate industrial remediation.

Hyper-Scale Training and Multi-Paradigm Corpora. Future work will expand training corpora to hybrid architectures combining synchronous gRPC/REST invocations with asynchronous Kafka/MQTT event meshes. To support hyper-scale enterprise graphs ($|V| > 10^5$), we plan to integrate mini-batch subgraph sampling (GraphSAINT [88] and layer-dependent HGT sampling [68]) into the training pipeline.

Future Directions: Distributed AI and Automated Remediation. We envision three major research extensions: (1) modeling distributed AI serving clusters (e.g., vLLM, DeepSpeed) to mitigate straggler cascades and reduce GPU idle power dissipation; (2) measuring hardware energy directly via RAPL and NVML to empirically quantify the joules saved by avoiding live staging chaos sweeps; and (3) advancing from predictive risk gating to prescriptive architectural synthesis, automatically generating pull requests that introduce circuit breakers, replica scaling, and tuned QoS contracts to eliminate single points of failure.

9. Conclusion

This work introduced **Software-as-a-Graph (SaG)**, a pre-deployment Static System Analysis framework that bridges Architecture-as-Code manifests and heterogeneous graph representation learning for dependability forecasting in complex distributed systems. By modeling distributed architectures as typed multigraphs encompassing applications, middleware topics, shared brokers, and hardware nodes, SaG predicts cascading failure criticality and provides standards-compliant ISO/IEC 25010 diagnostic attribution directly within CI/CD pipelines without executing code or deploying runtime telemetry.

Our extensive empirical evaluation across a twelve-scenario inductive benchmark and five authentic open-source distributed systems establishes four principal findings: (1) **Value of Typed Heterogeneity:** Relational typing provides a decisive architectural inductive bias for zero-shot transfer across complex distributed topologies. Under rigorous substrate, depth, and training parity, the relation-specific Heterogeneous Graph Transformer (`HGT-QoS`) achieves a statistically verified +0.114 lead in rank correlation over homogeneous attention ($\rho = 0.695$ vs. 0.581, won in 11 of 12 folds, $p = 0.0122$), replicated under capacity-matched controls. (2) **Fidelity on the Active Cascade Stratum:** In complex systems characterized by substantial zero-inflation from architectural redundancy and dead-end services, `HGT-QoS` retains approximately 60% of its ranking power ($\rho_{>0} = 0.407$), whereas structural centrality heuristics collapse to $\rho_{>0} = 0.183$ (retaining only 32%). Furthermore, `HGT-QoS` significantly outperforms structural baselines on top-K critical-set identification ($F_1@K = 0.507$ vs. 0.353, $p = 0.034$), successfully isolating high-risk failure spreaders across complex multi-hop paths. (3) **Architectural Domain Boundaries:** Zero-shot transfer to authentic open-source systems succeeds robustly across asynchronous pub-sub event meshes (e.g., ROS 2 Autoware $\rho_{>0} = +0.549$, Home Assistant $\rho_{>0} = +0.564$), where message-passing captures multi-topic fan-out mechanics. Conversely, performance inverts on synchronous RPC call trees (Cloud Microservices $\rho_{>0} = -0.314$), revealing an out-of-domain shift between asynchronous event meshes and vertical invocation hierarchies that motivates multi-paradigm training corpora. (4) **Computational Sustainability via Infrastructure Avoidance:** Evaluating dependability statically at manifest-time eliminates the substantial energy, carbon emissions, and server costs of continuously provisioning and tearing down staging clusters for dynamic chaos testing. Within the static pipeline, neural forward inference is exceptionally sustainable, executing in 56.2 ms on commodity hardware ($< 0.02\%$ of pipeline latency) and scaling linearly with edge count. The computational bottleneck resides entirely in deterministic all-pairs reachability ($O(|V|^2 + |V||E|)$), demonstrating that AI inference is computationally negligible in sustainable CI/CD gating.

To operationalize these insights, SaG provides a Dual-Engine Consensus Protocol that unifies the pattern recognition of graph attention with the deterministic guarantees of structural analysis. Future extensions will expand training corpora to hybrid synchronous–asynchronous distributed architectures, integrate mini-batch subgraph sampling algorithms (such as GraphSAINT) to scale model training to hyper-scale software systems ($|V| > 10^5$), and advance from predictive risk gating to prescriptive automated remediation.

Declarations

CRediT Authorship Contribution Statement. Ibrahim Onuralp Yigit: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision.

Declaration of Competing Interest. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Supplementary Material. Parameter sensitivity analyses (OFAT, Morris screening, threshold and normalization sweeps, zero-inflation diagnostics, and HGT attention distributions) are provided in the online supplementary document (Sections S1–S8).

Data Availability. The complete replication package—including synthetic datasets, generator configurations, simulation harnesses, real-world adapters, model checkpoints, and analysis scripts—is openly available on

Zenodo under DOI 10.5281/zenodo.14922108 [90]. Synthetic datasets regenerate byte-identically from committed configurations. All 237 reported table values are deterministically verified against backing JSON artifacts via the included `reproduce/reconcile_manuscript.py` script.

Declaration of Generative AI and AI-Assisted Technologies. During the preparation of this work the authors used Anthropic's Claude to assist with LaTeX typesetting and improve readability. After using this tool, the authors reviewed and edited the content and take full responsibility for the published article. No generative AI tool was used to design the study, analyze research data, or generate experimental results: all figures and tables are rendered deterministically from replication package artifacts.

References

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, Science Robotics 7 (66) (2022) eabm6074.

[2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: Proc. 6th Int. Workshop on Networking Meets Databases (NetDB), 2011.

[3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).

[4] OASIS, MQTT version 5.0, OASIS Standard, `https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html` (accessed 9 September 2026) (2019).

[5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: Present and Ulterior Software Engineering, Springer, 2017, pp. 195–216.

[6] S. Newman, Building Microservices: Designing Fine-Grained Systems, O'Reilly Media, 2015.

[7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, ACM Computing Surveys 35 (2) (2003) 114–131.

[8] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, Physical Review E 66 (2002) 065102(R).

[9] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, Nature 464 (2010) 1025–1028.

[10] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, Nature 406 (2000) 378–382.

[11] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, IEEE Transactions on Dependable and Secure Computing 1 (1) (2004) 11–33.

[12] L. Bass, P. Clements, R. Kazman, Software Architecture in Practice, 3rd Edition, Addison-Wesley, 2012.

[13] C. Calero, M. Piattini (Eds.), Green in Software Engineering, Springer, Cham, Switzerland, 2015. `doi:10.1007/978-3-319-08581-4`.

[14] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, Communications of the ACM 63 (12) (2020) 54–63. `doi:10.1145/3381831`.

[15] L. Lannelongue, J. Grealey, M. Inouye, Green algorithms: Quantifying the carbon footprint of computation, Advanced Science 8 (12) (2021) 2100707. `doi:10.1002/advs.202100707`.

[16] R. Verdecchia, J. Sallou, L. Cruz, A systematic review of Green AI, WIREs Data Mining and Knowledge Discovery 13 (4) (2023) e1507. `doi:10.1002/widm.1507`.

[17] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).

[18] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).

[19] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS), 1998, pp. 68–78.

[20] W. Cunningham, The WyCash portfolio management system, in: Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), 1992, pp. 29–30.

[21] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, Journal of Systems and Software 101 (2015) 193–220.

[22] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR), 2009, pp. 255–258.

[23] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, IEEE Software 35 (3) (2018) 56–62.

[24] SonarSource, Clean as you code, SonarQube documentation, `https://docs.sonarsource.com/sonarqube-server/latest/core-concepts/clean-as-you-code/introduction/` (accessed 9 September 2026) (2024).

[25] T. J. McCabe, A complexity measure, IEEE Transactions on Software Engineering SE-2 (4) (1976) 308–320.

[26] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, IEEE Transactions on Software Engineering 20 (6) (1994) 476–493.

[27] N. Fenton, J. Bieman, Software Metrics: A Rigorous and Practical Approach, 3rd Edition, CRC Press, 2014.

[28] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, IEEE Software 33 (3) (2016) 35–41.

[29] L. C. Freeman, A set of measures of centrality based on betweenness, Sociometry 40 (1) (1977) 35–41.
[30] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, Computer Networks and ISDN Systems 30 (1–7) (1998) 107–117.
[31] U. Brandes, A faster algorithm for betweenness centrality, Journal of Mathematical Sociology 25 (2) (2001) 163–177.
[32] M. E. J. Newman, Networks: An Introduction, Oxford University Press, 2010.
[33] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 9244–9255.
[34] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 19620–19631.
[35] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish–subscribe systems, in: Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE), 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.
[36] R. C. Cheung, A user-oriented software reliability model, IEEE Transactions on Software Engineering SE-6 (2) (1980) 118–125.
[37] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, Performance Evaluation 45 (2–3) (2001) 179–204.
[38] A. Immonen, E. Niemelä, Survey of reliability and availability prediction methods from the architectural perspective, Software and Systems Modeling 7 (1) (2008) 49–65.
[39] S. Becker, H. Koziolek, R. Reussner, The Palladio component model for model-driven performance prediction, Journal of Systems and Software 82 (1) (2009) 3–22.
[40] G. Franks, T. Al-Omari, M. Woodside, O. Das, S. Derisavi, Enhanced modeling and solution of layered queueing networks, IEEE Transactions on Software Engineering 35 (2) (2009) 148–161.
[41] J. Delange, P. H. Feiler, Architecture fault modeling with the AADL error-model annex, in: 2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA), IEEE, 2014, pp. 361–368. doi:10.1109/SEAA.2014.20.
[42] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019.
[43] Y. Gan, M. Liang, S. Dev, D. Lo, C. Delimitrou, Sage: Practical and scalable ML-driven performance debugging in microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2021.
[44] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS), 2020.
[45] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: Proc. IEEE/ACM Int. Symposium on Quality of Service (IWQoS), 2021.
[46] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2022.
[47] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2023.
[48] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, IEEE Transactions on Software Engineering 22 (10) (1996) 751–761.
[49] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: Proc. 27th Int. Conf. on Software Engineering (ICSE), 2005, pp. 580–586.
[50] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE), 2007.
[51] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, IEEE Transactions on Software Engineering 33 (1) (2007) 2–13.
[52] V. Bushong, D. Das, A. Al Maruf, T. Cerny, Using static analysis to address microservice architecture reconstruction, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2021. doi:10.1109/ASE51524.2021.9678749.
[53] S. Schneider, A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, R. Scandariato, D. Taibi, Comparison of static analysis architecture recovery tools for microservice applications, arXiv preprint (2024). arXiv:2412.08352, doi:10.48550/arXiv.2412.08352.
[54] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.
[55] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley, 2010.
[56] L. Chen, Continuous delivery: Huge benefits, but challenges too, IEEE Software 32 (2) (2015) 50–54.
[57] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
[58] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and

software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).
[59] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.
[60] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, Nature Machine Intelligence 2 (2020) 317–324.
[61] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM), 2019, pp. 559–568.
[62] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
[63] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2017.
[64] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 1024–1034.
[65] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2018.
[66] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: Proc. European Semantic Web Conference (ESWC), 2018, pp. 593–607.
[67] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: Proc. The Web Conference (WWW), 2019, pp. 2022–2032.
[68] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proc. The Web Conference (WWW), 2020, pp. 2704–2710.
[69] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: Proc. The Web Conference (WWW), 2020, pp. 2331–2341.
[70] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference, Morgan Kaufmann, 1988.
[71] G. Beliakov, A. Pradera, T. Calvo, Aggregation functions: A guide for practitioners, Studies in Fuzziness and Soft Computing 221 (2007).
[72] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, IEEE Transactions on Systems, Man, and Cybernetics 18 (1) (1988) 183–190.
[73] G. H. Hardy, J. E. Littlewood, G. Pólya, Inequalities, 2nd Edition, Cambridge University Press, 1952.
[74] U.S. Department of Defense, MIL-STD-498: Software development and documentation, Military standard, U.S. Department of Defense (1994).
[75] R. C. Martin, Agile Software Development: Principles, Patterns, and Practices, Prentice Hall, 2003.
[76] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
[77] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: Proc. 25th Int. Conf. on Machine Learning (ICML), 2008, pp. 1192–1199.
[78] Team SimPy, Simpy: Discrete event simulation for Python, Software, `https://simpy.readthedocs.io` (accessed 9 September 2026) (2020).
[79] International Organization for Standardization, ISO/IEC 25022:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of quality in use, Tech. rep., International Organization for Standardization (2016).
[80] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: Proc. ACM/IEEE 9th Int. Conf. on Cyber-Physical Systems (ICCPS), 2018, pp. 287–296.
[81] Google Cloud Platform, Online boutique: A cloud-native microservices demo application, Software, `https://github.com/GoogleCloudPlatform/microservices-demo` (accessed 9 September 2026) (2024).
[82] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, IEEE Transactions on Software Engineering 47 (2) (2021) 243–260.
[83] Home Assistant Community, Home assistant: Open source home automation that puts local control and privacy first, Software, `https://www.home-assistant.io/` (accessed 9 September 2026) (2024).
[84] Linux Foundation LF Edge, Edgex foundry: An open, vendor-neutral edge iot middleware platform, Software, `https://www.edgexfoundry.org/` (accessed 9 September 2026) (2024).
[85] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.
[86] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.
[87] C. Spearman, The proof and measurement of association between two things, American Journal of Psychology 15 (1) (1904) 72–101.
[88] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, V. Prasanna, GraphSAINT: Graph sampling based inductive engine, in: Proc. International Conference on Learning Representations (ICLR), 2020.
[89] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, J. Dean, Carbon emissions and large neural network training, arXiv preprint arXiv:2104.10350 (2021). `doi:10.48550/arXiv.2104.10350`.
[90] I. O. Yigit, F. Buzluca, [dataset] software-as-a-graph: Replication package (datasets, generator configurations, simulation harnesses, model checkpoints, and analysis scripts), `https://doi.org/10.5281/zenodo.14922108` (2026). `doi:10.5281/zenodo.14922108`.